Essence C

A new programming language to advance link-time optimizations by faciliating the continuum of static and dynamic modules.

David Callanan

21444104

Final Year Project – 2026

B.Sc. Single Honours in Computational Thinking

Department of Computer Science

Maynooth University

Maynooth, Co. Kildare

Ireland

A thesis submitted in partial fulfilment of the requirements for the

B.Sc. Single Honours in Computational Thinking.

Supervisor: Dr. Phil Maguire

Contents

Declaration 5

Acknowledgements 6

1 Introduction (930) 2

1.1 Overview 2

1.2 Motivation 2

1.3 Methodology 3

1.4 Success Criteria 3

1.5 Use of Generative AI 4

2 Technical Background (1200) 5

2.1 Literature Review 5

2.1.1 Static vs dynamic dispatch 5

2.1.2 Static vs dynamic linking 5

2.1.3 Module loading in the Linux kernel 6

2.1.4 Inlining and IR-level linking 8

2.2 Technical Material 8

2.2.1 LLVM Intermediate Representation 8

2.2.2 Link-time optimizations 8

3 The Problem (900) 10

3.1 Problem Statement 10

3.2 Functional Requirements 10

3.3 Non-Functional Requirements 10

4 The Solution (2110) 11

4.1 Design Elements 11

4.1.1 Choice of Language 11

4.1.2 Unification of Modules and Classes 11

4.1.3 Map-Enum Equivalence 12

4.1.4 Body-Map Unification 13

4.1.5 Type-Value Unification 13

4.1.6 Additional benefit of inlining: interacting with existing languages without indirection. 14

4.1.7 Associativity 14

4.1.8 Module Resolution 15

4.1.9 Constant By Default 16

4.1.10 No Standard Library 16

4.2 Implementation 17

4.2.1 Build Environment 17

4.2.2 Parsing 17

4.2.3 Parser Trace Explorer 18

4.2.4 Defensive Programming 19

4.2.5 Mutability Support 19

4.2.6 Debugging Primitives 20

4.2.7 Bit-width Analysis 20

4.2.8 Underlying Type Tracking 21

4.2.9 Singletonish Optimization 21

4.2.10 Static Instance Analysis 21

5 Evaluation (930) 22

5.1 Manual Snapshot Testing 22

5.2 GenAI Comprehension Testing 22
5.3 Benchmarking 23
5.4 Validation 23

6 Conclusion (930) 25

6.1 Overall Assessment 25
6.2 Contribution to state-of-the-art 25
6.3 Limitations 25
6.4 Future Work 26

A Record of Meetings 28

B LLM Comprehension Testing 29

B.1 Documentation Prompts 29
B.2 Generated Documentation PDF 44

C Screenshots 59

D Source Code 60

E LLM Prompts 61

F Interim Report 62

Declaration

I hereby certify that this material, which I now submit for assessment on the program of study as part of the B.Sc. Computational Thinking qualification, is entirely my own work and has not been taken from the work of others – save and to the extent that such work has been cited and acknowledged within the text of my work.

I hereby acknowledge and accept that this thesis may be distributed to future final year students, as an example of the standard expected of final year projects.

Signed: _____ Date: _____

Acknowledgements

A massive thanks to Dr. Phil Maguire, friends, and peers for inflicting levels of motivation I might never reciprocate.

Some brief gratitude to Anthropic for their powerful GenAI models, though other models were preferred due to cost reasons.

Abstract

The logic of computer software is primarily expressed using programming languages, but the balance between versatile and efficient programming languages remains a major challenge. Changes to the execution environment of a program can impact performance, or worse, cause incompatibilities that require major refactoring. Developers often maintain multiple branches of a codebase to keep performance to acceptable levels. One particular difference between execution environments is whether individual modules are statically linked at compile-time or dynamically resolved at runtime. In this research project, we propose a prototype programming language called "Essence C" that better supports the continuum of static and dynamic modules, by allowing the programmer to write code *once*, that is agnostic to the linking strategy. This is achieved by designing unique language features that are inspired by *zero-cost abstraction* principles. By taking advantage of LLVM's intermediate representation, it is possible to advance the state-of-the-art of link-time optimizations across module boundaries. The performance of these optimizations is assessed by benchmarking four simple hardware operations in a proof-of-concept operating system kernel, consisting of *C*, *Essence C*, *NASM*, and *LLVM IR*. The ability for large language models to assist with using *Essence C* is also evaluated.

Chapter 1 – Introduction (930)

1.1 Overview

This project is concerned with the research and development of a novel programming language, Essence C, that facilitates the continuum of static and dynamic modules in low-level software development. The prototype henceforth discussed is accomplished by designing unique language features and implementing bespoke optimizations pursuant to these features. Kernel development is used as a case study throughout the research phase and evaluation phase.

1.2 Motivation

Software development is becoming very popular, with graduates now unable to find jobs due to a massive spike in educational resources allocated to this field. With the use of LLM agents gaining traction, keeping a codebase clean may be harder than ever before. There is now serious competition in programming language design due to two reasons. Firstly, an LLM's ability to navigate a codebase effectively depends on the structure imposed directly by the language, given the reduced structural input from the programmer. Techniques that minimize duplicated effort in a codebase is expected to minimize context rot and reduce the agentic workflow bills. Secondly, agents are at risk of optimizing their development in highly trained-on environments (e.g. JavaScript, web-based, and *React.js* environments), leaving less-prevalent low-level languages behind unless the agent can learn from on-the-fly tool calls. Highly structured languages with a superb type system, high cohesion of language concepts, and excellent documentation may make up for the lack of training. An effective module system alongside other novel language features has the potential to transform the way humans and agents reason about their software. This includes the desire to be able to freely architect software for organizational, abstractional and future-proofing purposes without compromising performance or compatibility. Developers are often fed up with having to rewrite code when environmental changes are bestowed upon them.

This thesis focuses on one key area where such increased flexibility is desirable: linking and dispatch strategies for modules and objects alike.

1.3 Methodology

To address the objectives of this research, a design-science methodology is employed with a focus on continued interation and evaluation of "Essence C". This approach was selected because the project is concerned with the implementation of a novel programming language that advances the flexibility and performance of modules in software development, and thus requires a design and implementation process that is informed by ongoing evaluation.

The structure of this thesis is organized to reflect this methodology. First, Chapter 2 reviews existing literature, both theoretical and technical, focusing on how kernels optimize static calls and how the LLVM toolchain is effective for programming language development. Next, Chapter 3 summarizes the key aspects of the problem, including the functional and non-functional requirements. Then, Chapter 4 considers separately the design elements and implementation details of the solution. Penultimately, Chapter 5 evaluates the success of the project in the areas of snapshot testing, performance benchmarking, and GenAI comprehension. Finally, Chapter 6 concludes with an overall assessment, limitations, and future work.

1.4 Success Criteria

Success is evidenced by the following deliverables:

(1) A functional prototype compiler that implements the designed module system;
(2) Documentation of the module system and other core language features;
(3) A simple IDE extension that provides syntax highlighting to the programmer;
(4) A body of research into the continuum of static and dynamic modules in software development, with kernel development as a case study;
(5) An implementation of link-time optimization techniques that leverage the designed module system;
(6) An evaluation of the effectiveness of these optimizations through the benchmarking of a prototype operating system kernel;
(7) A monorepo containing extensive git history, demonstrating ongoing development progress;
(8) A final thesis (that you are reading) detailing the research, design, implementation and evaluation of the project.

Some considerations will be out of scope for this project, such as the performance of compilation, the level of detail of compilation errors, and the implementation of standard library functions and advanced type system features.

1.5 Use of Generative AI

Two notable GenAI tools have been used to assist with the research and development of this project.

(1) The first is t3.chat, a web application that consolidates numerous large language models into a streamlined chat interface [1]. The user can experiment with different models to understand which models are best suited for the task at hand. Both research and programming tasks have benefited from the use of this tool. The tool was primarily used to steer the work in the right direction, and to identify the concepts, libraries and resources through which further research could be conducted. For transparency, all LLM prompts have been included in **Appendix C**. This appendix was generated programatically by using the JSON export facility and filtering relevant chats.

(2) The second is the tab-complete feature of GitHub Copilot. This tool has encouraged rapid development and iteration through its behaving as a glorified autocomplete. From experience, the tool's context is severely limited, forcing the developer to have genuine knowledge of the codebase and its architecture. This is beneficial for projects that require significant unique input from the developer, in contrast to rehash projects that benefit more directly from LLM assistance. The latest iteration of this feature allows completions for lines that are away from the cursor, which has been a game-changer.

Some experimentation with GitHub Copilot agent mode was conducted, with notable issues identified. Firstly, the development speed was impeded by slow model response times, lack of funds, and internet connectivity issues. Secondly, there was a great difficulty in referencing contributions made by this tool due to (1) tool calls that modify the codebase directly and (2) significant implicit pollution of the context window as the agent navigates the respository. As such, with an exception for debugging, usage of this tool was suspended.

The logo on the cover page was generating using *Nano Banana Pro*.

Chapter 2 – Technical Background (1200)

2.1 Literature Review

2.1.1 Static vs dynamic dispatch

There are two primary ways for methods of objects (or equivalent constructs) to dispatch calls: static dispatch or dynamic dispatch. When the exact implementation of a method is known at compile-time, static dispatch is typically favoured, involving a direct assembly call to the method. Since the method is known at compile-time, the compiler can use immediate addressing mode, which is most efficient. In contrast, when the exact implementation of a method is not decided until runtime (or if the compiler cannot prove otherwise), dynamic dispatch is required, which involves an indirect call through a vtable or similar structure.

2.1.2 Static vs dynamic linking

There is much overlap in the literature between dispatch techniques and linking techniques. Linking focuses on larger units of code, often referred to as modules or libraries, as opposed to objects. This is crucial in C where there is no inbuilt notion of static vs dynamic dispatch. Rather, C programmers often use seperate compilation units to allow the user to substitute implementations. Unfortunately, architectural limitations often result in the use of the unfavourable PIMPL idiom which can tank performance, but I digress.

To ditto the above, there are two primary ways for compilation units to be linked: static linking and dynamic linking. When the exact source code (implementation) of a module is known at compile-time, static linking is typically favoured, involving the direct inclusion of the module's code into the final executable. Then, the compiler can use immediate addressing to invoke functions or access data from this module. In contrast, when the exact source code of a module is not decided until runtime, dynamic linking is used which typically requires a Procedure Linkage Table (PLT) and Global Offset Table (GOT).

Various techniques are already employed to optimize dynamic linking:

(1) Some architectures use the "relaxation" technique to replace indirection at runtime by modifying the assembly instructions.
(2) Good compilers can perform PLT rewriting at compile-time if the nature of the calls is known.
(3) Bespoke replacement (e.g. Linux kernel static calls).

However, a primary concern remains: Various optimization techniques such as inlining become impossible due to this indirection.

2.1.3 Module loading in the Linux kernel

It was found that the potential inefficiencies of dynamically loaded modules were already considered by the Linux kernel developers, and boutique infrastructure was developed to partially mitigate this.

The Linux kernel benefits from their "static_call" system, which uses a combination of C macros, custom compilation/linking steps, and runtime logic to patch function calls to be more efficient. Any calls that may need to be dynamically swapped at runtime can engage with the "static_call" system to improve performance and automate the repetetive assembly logic. The process is summarized below.

Macro system

Each static-call function is implemented using the "DEFINE_STATIC_CALL" macro, which takes in (1) a handle name, and (2) a pointer to the initial implementation for this handle. This macro is used at the top-level. Then, the programmer can use the "static_call_update" macro at any point, taking in the same parameters, to alter the implementation for a particular handle at runtime. Alas, the programmer can use the "static_call" macro to invoke the handle like a traditional function. Here is some example pseudo-code:

```
void implementation_a(int value) { ... }
void implementation_b(int value) { ... }

DEFINE_STATIC_CALL(foo, implementation_a);

void main() {
  static_call(foo)(27);
```

```
  static_call_update(foo, implementation_b);
  static_call(foo)(27);
}
```

The “static_call” macro also uses the inline assembly “.pushsection” and “.popsection” directives to temporarily relocate the code to a unified object section that can be picked up in the custom compilation step.

A crucial takeaway is that the lack of direct language support has forced the need for a custom macro setup. Understandably, programmers in different codebases may not be so adamant to commit resources to such a setup in the same way the Linux kernel developers have.

Custom compilation step (objtool)

The kernel needs to remember where the call sites are when the implementation is swapped at runtime. To achieve this, the kernel includes an extra build step that investigates the aforementioned object section to locate all relevant call sites. It then builds up a table that can be inspected at runtime to ensure no call sites are missed. There are many more intricacies that are not mentioned and have not been studied in too much detail.

If you explore the source code for the Linux kernel, you will come across the static_call_types.h file, containing the following:

```
/*
 * The static call site table needs to be created by external tooling (objtool
 * or a compiler plugin).
 */
struct static_call_site {
    s32 addr;
    s32 key;
};
```

Trampoline solution

The easy solution to alter the implementation of a function at runtime is to use a trampoline (an extra dynamic jump instruction that is resolved at runtime). This is slow and is merely a backup option.

Insert diagram.

Inlining solution

Great news, inlining refers to the trampoline call, not inlining the function body itself.

(Well not so great news, but at least there is more merit to this project).

Spectre

Since the infamous Spectre exploit in 20XX,

2.1.4 Inlining and IR-level linking

2.2 Technical Material

2.2.1 LLVM Intermediate Representation

LLVM was found to be a practical compilation target for low-level programming languages due to (1) its target-independent code generation [2] and (2) its integration capabilities with existing programming languages thanks to ABI interoperability (including some recent improvements [3]). Furthermore, the extensive optimization passes shared amongst languages targetting LLVM [4] should make *Essence C* directly comparable with existing languages, allowing benchmarks to be solely influenced by the additional optimization techniques employed. For instance, C can be compiled with LLVM via the popular Clang compiler [5]. It was also noticed that LLVM has experimental support for inline assembly [6], preventing the layer of indirection of a separate compilation unit (useful in kernel development [7]). Additionally, LLVM benefits greatly from intermodular optimizations at link-time [8], because its intermediate representation (IR) preserves the necessary higher-level constructs. Critically, the LLVM API exposes a powerful interface to programatically manipulate existing LLVM IR files [9] [10]. It follows that inlined compatibility wrappers can be generated to minimize optimization barriers between compilation units originating from languages other than *Essence C*.

2.2.2 Link-time optimizations

Traditionally, many optimizations were performed at the language level, before the program was compiled to object files. For instance, inlining could not be performed at link-time because there was insufficient information in the object files to reason about the semantics of the program. While dedicated link-time optimizations were introduced, their scope

was limited for this reason. However, LLVM IR sits in between the level of abstraction of source code and object files, and maintains higher-level semantics. By keeping the program in this intermediate representation longer (and treating this as the "modern object file"), recent developments have made link-time optimizations more powerful, allowing, for instance, for inlining across module boundaries.

Chapter 3 – The Problem (900)

3.1 Problem Statement

How can we design and implement a new programming language that allows the programmer to easily switch data and logic from dynamic to static dispatch and subsequently benefit from link-time optimizations such as inlining across module boundaries? How can we evaluate that the programming language is effective for this purpose?

3.2 Functional Requirements

The language must support the majority of features available in the C programming language (except macros): numerical expressions, logical expressions, functions, structs, conditional statements, loops, pointers, etc. Then, the language must be capable of linking in with external compilation units, so that, for example, the C standard library functions can be used.

3.3 Non-Functional Requirements

The language must have freestanding viability so that it can be used in kernel development. The language must have unified language concept(s) that treat static vs dynamic linking of modules in the same way as static vs dynamic dispatch of objects. The language must allow the programmer to specify their intent to switch to a fully static build by substituting some simple keywords, without having to refactor the code.

Chapter 4 – The Solution (2110)

4.1 Design Elements

4.1.1 Choice of Language

The languages selected for this project were JavaScript and C++. While JavaScript has been my go-to language before classes were even a thing, the primary LLVM API is sadly exposed through C++. Other language bindings exist, but do not appear mature enough. It would not have been smart to gamble a project of this calibre on the stability of such bindings. JavaScript was still suitable for the parsing phase, as my pre-existing parsing library (called *blurp*) was already written in JavaScript. The speed of compilation is mostly irrelevant, as this does not impact the performance of the compiled artifacts. Thus minimal effort was wasted on such a triviality.

4.1.2 Unification of Modules and Classes

Languages have historically offered various features like classes, modules, packages, and so on, that have much overlap in their functionality. One of the core design decisions was to unify these elements into a single system that consists of a smaller unit called a "map" (which bears some resemblence with Rust- and Go-like structs), and a larger organizational unit called a "module". Most importantly, every module still uses a map for instance-management purposes, and so the same capabilities are available whether or not a program is split into more modules or not. This differs from languages like C, where the choice to have separate compilation units has an impact on the final program [ref]. The decision for this unification was inspired by Rust's philosophy of "zero-cost abstractions" [ref], an acknowledgement that organizational differences in a program that aid with design and iteration should not impact the performance of the final artifact.

4.1.3 Map-Enum Equivalence

Three features in programming languages were found to share an emergent goal: structs (or classes), functions (or methods), and enums (specifically, tagged unions).

Essence C unifies structs and functions into a single feature called a “map”. This takes inspiration from Pure Mathematics, where the pattern of notational definitions is often used for multiple purposes – functions ($f(x)$), operations ($x+y$), subscripts (x_n), accessors ($\star_G$). Little special treatment is given to these different cases, as all could be denoted with function-like syntax ($f(x), +(x, y), x(n), \star(G)$).

In Essence C, there are no struct accessors like “.first_name”. Rather, a struct is a “map” that takes in a symbol (e.g. “:first_name”) as its only parameter and returns the member from its internal storage. These semantics are not so bizarre, as the technique to store data alongside functions (often known as closures) is already commonplace. When a "map" has no virtual methods, its behaving as a struct results in the same low-level data representation.

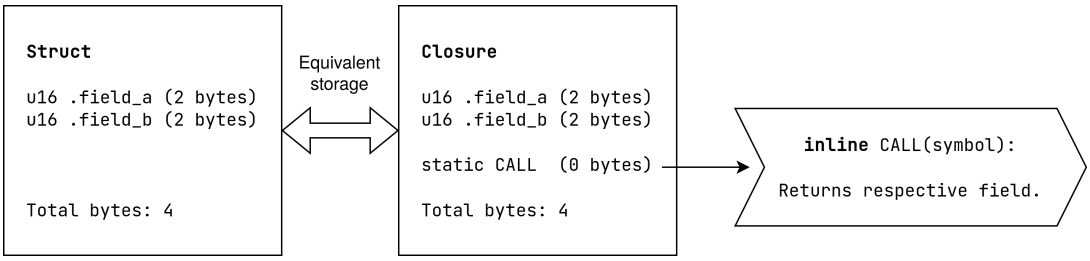

Figure 4.1: This diagram showcases the equivalent data representation between a traditional struct (left) and a field extraction closure (right).

In the same vein, tuples are implemented as syntactic sugar for maps, where positional syntax is translated into numeric symbol input. One notable outcome of this decision is that there is no concept of multiple function parameters; instead, there is a single parameter which reuses the existing tuple syntax to achieve multiple parameters. It’s maps all the way down.

Next, maps and enums are unified via a so-called Map-Enum Equivalence rule: Recall Rust-like enums (tagged unions) where a symbol (:foo_bar) is associated with some data (:foo_bar("hello")). A syntactical ambiguirty arises. Writing map:foo_bar("hello") may be interpreted as first calling the map with a symbol (an enum without data) and then calling the resulting map with ("hello"). It may also be interpreted as calling the map with the fully populated enum (:foo_bar("hello")). The solution is to unify these two approaches, by demanding that all maps obey map-enum equivalence, that is, that calling a map with a symbol and then another map is equivalent to calling the map with the fully populated enum. This bears partial resemblance with currying: Populated enums

can now be understood as pre-curried symbols instead of an entirely distinct concept.

In summary, the vast majority of code written by the programmer now comes from maps and symbols!

4.1.4 Body-Map Unification

4.1.5 Type-Value Unification

Old-fashioned languages treat types and values as entirely distinct. In the traditional sense, a type offers two functions:

(1) It details the runtime representation of the data, i.e. the possible runtime values that may be encoded and how many bits of data are needed.
(2) It provides a constraint to the programmer themself if they later modify the code, preventing the programmer from using values that the code is not built for.

It turns out that it may be appropriate for these two intents to diverge. This results in two separate notions of a type:

- **Logical Type:** A set of allowable values. A constraint of what theoretical values are permissible, i.e. subject the programmer's intended business logic.
- **Encoding Type:** A bijection between a set of values and a set of bit patterns. This allows logical types to be designed and abstracted independent of the encoding used.

To achieve this distinction, the syntax for logical types and values are unified by treating everything as a constraint. A value is just a singleton constraint, i.e. one possible value. Types are then merged using a binary reduction algorithm such that the final type meets all constraints. Then, an appropriate type encoding is determined that works in harmony with the logical type.

With this in mind, there is no syntactic differentation between types and values, and there is no syntax to give a variable a type. For instance, as illustrated in Figure 4.2, we can assign a variable to a type, a value, or both. Semantically, however, the compiler will bail if it realizes there is no single runtime representation available.

```
; non-singleton assignments

x := u64;

y := {
    :name *u8;
    :age u16;
    :is_happy bool;
};

z := enum {
    :option_a;
    :option_b;
};
```

```
; singleton assignments

a := 10;

b := {
    :name "David";
    :age 23;
    :is_happy :false;
};

c := :option_a;
```

```
; pairwise constraint merging

p := u64 10;

q := {
    :name *u8 "David";
    :age u16 23;
    :is_happy bool :false;
};

r := enum {
    :option_a;
    :option_b;
} :option_a;
```

Figure 4.2: How variables can be assigned to a non-singleton type, a singleton type (value), or a pairwise merging of two types.

4.1.6 Additional benefit of inlining: interacting with existing languages without indirection.

4.1.7 Associativity

In most programming languages, addition and subtraction are treated as unary operations in an additive batch, whereras operations like multiplication and division are instead treated as binary operations in a multiplicative tree. However, it turns out that any associative operation (such as multiplication) can be written in the former form, where operations in the same family (such as division) control the unary effect via an operator that prefixes the operand. As such, a consistent syntax was employed in this language, allowing both the unary and binary approach for all associative operations and their respective intra-family operations. This includes the following families:

- Addition and subtraction (like existing languages);
- Multiplication and division;
- Logical AND;
- Logical OR;

The syntax was designed to follow the crystal/pistol pattern, some terminology devised to mean the following:

- Crystal: A unary batch of operator/operand pairs (an extension of how existing languages handle addition and subtraction).
- Pistol: A binary tree of operations (like how existing languages handle multiplication and division).

More details on this pattern can be gathered from my article on **boundly associative operations**, my article on **making complicated if statements cleaner**, and my YouTube video on **rethinking operation order**.

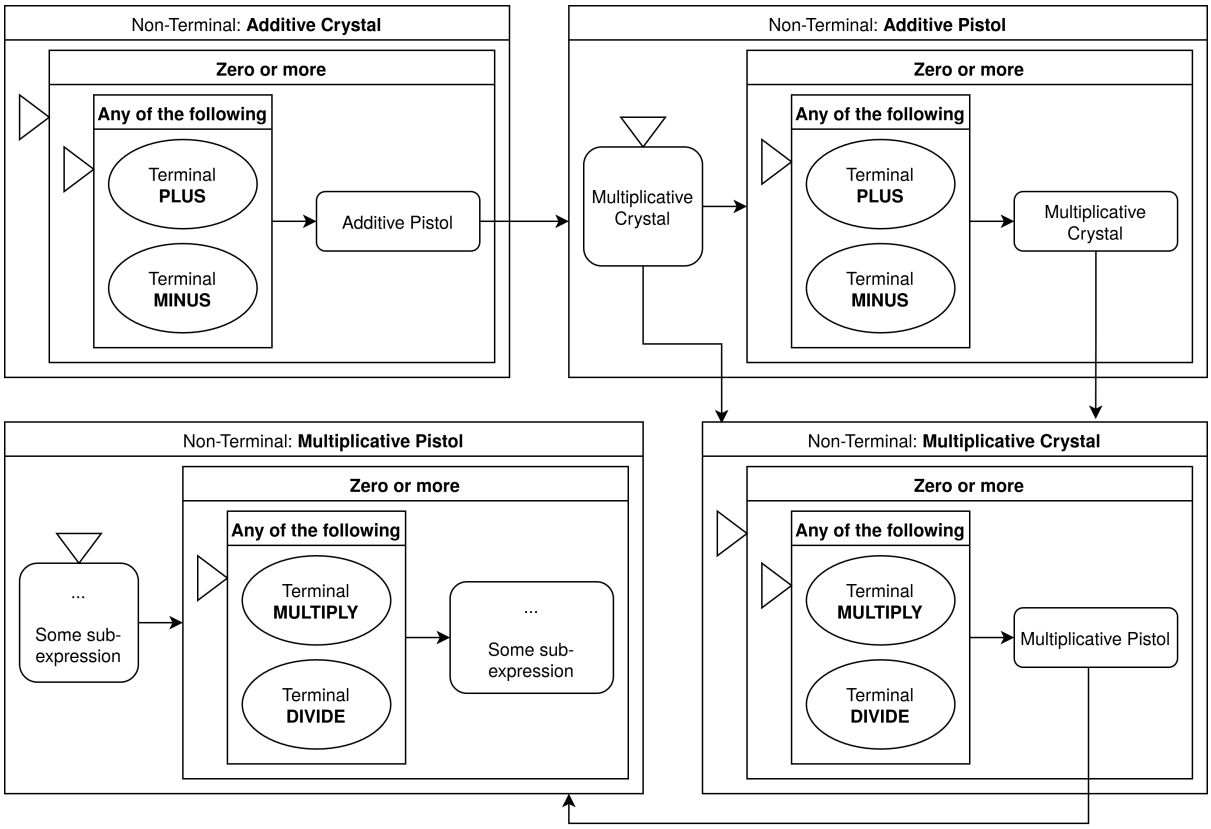

Figure 4.3: A simplified extract of some of the associativity and precedence rules for numerical expressions.

4.1.8 Module Resolution

Some (debatably legacy) approaches to module resolution are:

- Use the filesystem alongside symlink functionality to ensure access to the intended modules. This is effective for source code because a single build of a project does not need to switch between versions of source code (if some source code *might* be used, it is *always* included). While multiple instances of source code may be simultaneously deployed, the typing information is fixed. However, many developers consider using the filesystem to be a lot of hassle. Instead, this language takes the stance that if a built-in feature that all developers use (a filesystem) can handle module resolution for types and source code, then why re-invent the wheel. Thus, Essence C promotes the symlink approach for this purpose. This has two consequnences:
  (1) Language-level Ownership: A module takes ownership of type dependencies. While a module may elect to use an external package management solution,

symlinks, etc. to resolve the interfaces, by the time the dependencies reach the compiler, the compiler treats the interface as under the ownership of that module. This is favourable to those who follow the dependency inversion principle, but may feel foreign to those who delegate type ownership to external systems.

(2) Monorepos or "Physical" Ownership: Those who like to use monorepos or like to own their dependencies physically (that is, not just within the language, but as a non-functional business rule), will have no trouble baking copies of dependencies into their repo. This is a huge reason why many developers (including myself) switched over to the "pnpm" package manager for JavaScript projects.

- Paths: Compare the common use of paths to the Plan 9 operating system. Using paths is arguably messy as it is easy to pollute the global namespace.

One must distinguish instance resolution, source code resolution, and type resolution.

As Essence C wants serious static analysis of modules, passing instances is the only solution for the instance part. Spaghetti is avoided by using the factory approach as detailed in the module system.

However, the language takes a balanced approach when it comes to soruce code and type resolution, allowing you to use internal languages features to control the propogatoin of such dependencies. This allows a library to put the burden of populatoin on the user of the library. While this transfers a certain burden of ownership, is unlocks a flexibility that allows developers to not be inmpacted by personal resolution preferences of developers. This is a trade-off that requires further contemplation.

4.1.9 Constant By Default

4.1.10 No Standard Library

With kernel development as the primary target audience for this language, the language should be freestanding and not have any external dependencies. In kernel development, it does not make sense to have built-in functions for most features, as the kernel developer is expected to implement all features themselves (printing, keyboard input, memory allocation, etc.) by communicating directly with the hardware using assembly instructions. Thus, the language only externs one function, "puts" , for logging purposes. This function is easy enough to implement early on in kernel development, but the developer can also choose to use a no-op and avoid logging, if they so please.

Rather than a standard library, first class integration with LLVM IR and the C ABI

was implemented. This means the code can interact with any code written outside the language, preventing the need for the developer to migrate existing code to Essence C.

4.2 Implementation

4.2.1 Build Environment

Early on in development, I was adamment to ensure a fully reproducible build environment. All building is done using the terminal, so that the build process is independent from the editor used. The JavaScript portion of the project uses *Node.js* and *pnpm*, which are easy to install on different operating systems. Build scripts are then organized into *package.json* files. However, the C++ portion, consisting of a suite of tools (*clang*, *make*, *llvm*, etc.), is more difficult to set up consistently. The build scripts for this portion are organized using a Makefile, and the development environment itself is prepared using a Dockerfile. Some developer scripts were implemented in Bash to orchestrate the overall process between the JavaScript and C++ portions, the Docker layer, and the developer's own environment. The choice to use Bash means the developer must either use a Linux environment or a WSL environment if using Windows. A docker-in-docker strategy could theoretically have been used instead, requiring minimal platform-specific scripts, but this complexity was avoided. MacOS users may suffer as a result. Additionally, I recently switched to NixOS on my own system, and its philosophy was applicable here. NixOS allows you to write description-based software setups, which can act as reproducible development enviornments. Thus I set up a repository-level Nix script for those on NixOS who want to fast-track setting up their development environment. This makes both the build layer and host interaction layer highly reproducible.

4.2.2 Parsing

Prior to this project I had already developed a simple parsing library in JavaScript, which allows for:

(1) Parsing terminal rules using regular expressions.
(2) Forming non-terminal rules by combining other rules in the following ways (sufficient to develop any complex grammar):
    - "or": making a rule optional
    - "join": sequencing rules
    - "multi": repeating rules zero or more times
    - "opt": making a rule optional

- "lookAhead": peeking ahead to prevent ambiguity (seldom used)

(3) Map parsed data to custom structures using the "mapData" function.

Essence C was then parsed by handcrafting hundreds of parse rules using this library.

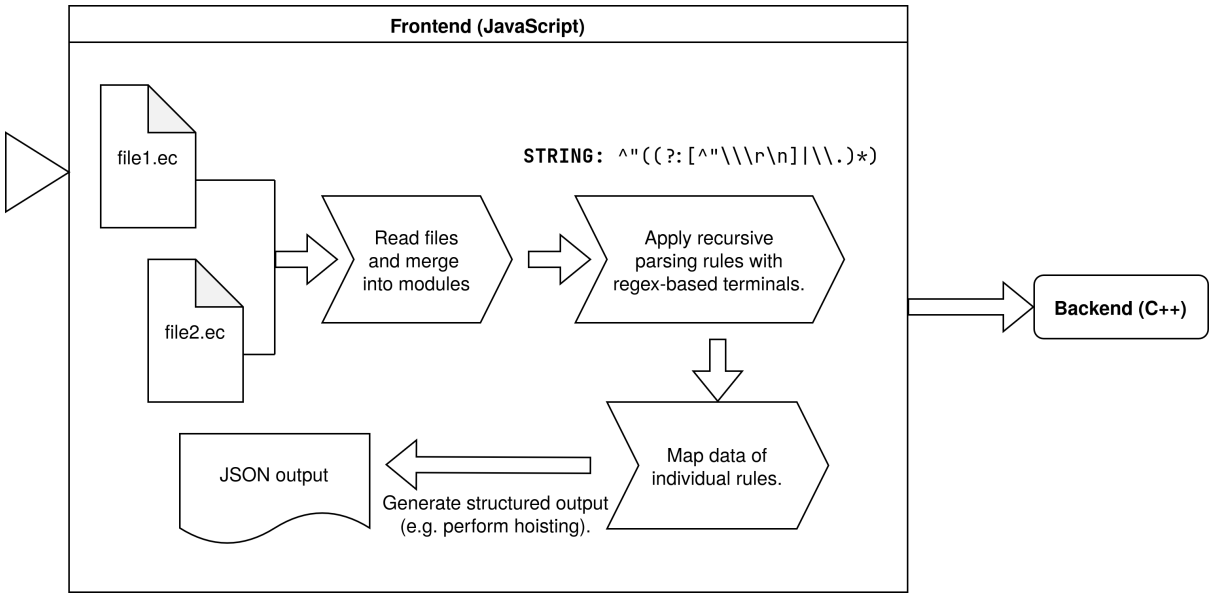

Figure 4.4: High-Level Frontend Architecture

4.2.3 Parser Trace Explorer

Parsing issues were frequently encountered during the development cycle, and manual debugging became infeasible as complexity increased. Hence, a *Parser Trace Explorer* was developed, consisting of a JSON trace logger in the compiler frontend, and a separate web-app tool for exploring these logs. A simple SolidStart + Tanstack Router tech stack was selected for this tool due to existing familiarity with this stack, and because of the straightforward requirements of the tool.

Since parsers try a vast number of rules and are inherently recursive, the logs quickly exceed $500,000$ lines for small source files. To deal with this, the logs are structured as a tree, and the user of the tool is expected to navigate this tree by selecting the precise rule attempts that the parser is supposed to succeed on, thus hiding irrelevant attempts. A dedicated lexer, in hindsight, would have reduced the exponential growth of the logs, but might not have easily parsed some of the unique language features (such as semicolons acting as both terminators and comments).

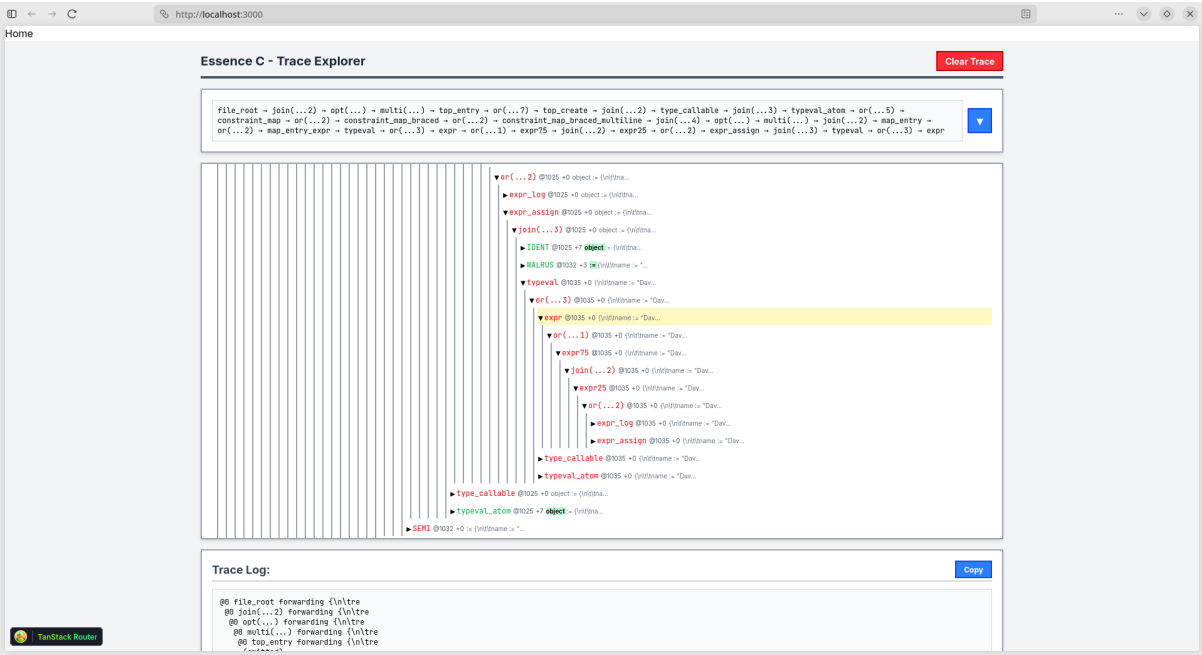

Figure 4.5: Parser Trace Explorer

4.2.4 Defensive Programming

As Essence C is designed with strict semantics, the backend has no need to recover from illegal states. As such, defensive programming is used, where the program is crashed with a useful error message whenever an invariant is violated. Such errors could originate from semantic violations in the user's code, or due to subtle bugs in the compiler itself; either way, the issues can be traced effectively. Naturally, not all subtleties can be pre-envisioned, so this was done on a best-effort basis to avoid compromising development speed.

4.2.5 Mutability Support

LLVM IR is in SSA form, where registers are immutable. One cannot reuse a register, and must instead create a new register for every mutation. Then, one takes advantage of the phi instruction to pick which register is the correct one at runtime. However, the alloca (stack allocation) model operates under the familiar mindset of mutability, as it works with raw memory addresses. Conveniently, it was observed that the standard mem2reg optimization pass does a fantastic job at converting stack allocations into SSA registers where legal, bypassing the tough development of a custom SSA mutability mechanism. Thus, variables are entirely implemented using stack allocations. This saved an estimated week of development time.

4.2.6 Debugging Primitives

While the primary mechanism of performing external communication is to link with external code via the C ABI (which may be LLVM IR), debugging functionality is often preferable within the language directly in case there are issues with the interoperability layer proper. Thus, two basic logging functions were implemented, "log" and "log_d". The former takes a null-terminated ascii string and prints this output in an implementation-specific manner. The latter takes an arbitrarily-sized value (like an integer or a byte array) and prints its raw binary representation (in 64-bit chunks), including both its hex representation, ascii representation, and decimal representation. The implementation of this functionality was chosen to be implementation-specific. In the prototype implementation, the external "puts" function was selected to implement this functionality, available in many environments. In case the environment does not support it, a dummy no-op implementation can be defined, effectively dropping any logs.

4.2.7 Bit-width Analysis

Determining the maximum bits needed to hold an integer is an important optimization. While LLVM may be able to detect certain constraints about numbers, much of the compiler information is lost at this point that a modern programming language might be interested in tracking. For example, software verification languages may keep track of exact min and max values for an integer type, which is tracked extensively as the type system navigates the codebase. For non-integer types, an optimal type system implementation might allow the user to use formal logic to keep track of the theoretically-smallest set of runtime possibilities (a provably accurate picture of the possible values that can be assigned at runtime, with no wasted space).

Only simple bit-tracking is performed in the prototype compiler. When a hardcoded integer constant is used, the number of bits is trivially calculated using the logarithm. Addition was more tricky to implement – one must recursively track the bits needed to hold an arbitrary sum of other integers. The first attempt of taking the maximum of the operand bits and adding one (to account for a potential carry) was insufficient for more than two operands. While an additional carry bit could be added for each additional operand, this estimate would be too conservative. The correct solution was to sum the maximum value each operand could take on (interpreted as data to avoid to intricacies of signed integers), and then take the logarithm of this sum. This can achieved easily in languages that support arbitrarily sized integers out-of-the-box. However, it was easier to implement an approximate solution in C++ by using the double data type, with the final result rounded up. While this sounds bizarre and inaccurate at first glance, recall

that floats are internally implemented using an exponent and mantissa. Thus, it is very similar to running the numbers through a logarithmic transformation, which would yield the final result directly. Furthermore, an epsilon value was included to deal with precision errors, favouring a more conservative bit estimate. It was not proven whether this overall approach is mathematically sound up to a particular bit count, but it can always be substituted if the need arises.

4.2.8 Underlying Type Tracking

4.2.9 Singletonish Optimization

A custom "singletonish" optimization was implemented that concerns itself with both the module optimizatoin goal (detecting something to be static) and the acknowledgement of the benefits of singleton types (by treating lonely values as singleton types). It also is meets the demands of the map enum equivalence, central to the design of the module system, as it allows different aspects of maps which are inherently static (e.g. the functional nature of maps) to be optimized to a form that behaves like traditional programming languages. This means the map-enum equivalence is not just theoretical, but acutually works.

4.2.10 Static Instance Analysis

Chapter 5 – Evaluation (930)

5.1 Manual Snapshot Testing

Throughout the development cycle, manual testing was conducted by compiling sample source files and inspecting the terminal output after running the final executable. This mirrors the snapshot testing philosophy commonly implemented in web-development projects (although accomplished in a less automated fashion here), where input/output consistency is checked before each commit to source control. Down the road, this strategy could be extended to a full test suite that prevents all sorts of regressions. One could take this a step further by checking consistency in the compiled LLVM IR code itself, but this is typically only done when a language reaches a pivotal level of maturity; the compiler is in too much flux at this point for this technique to have any merit.

Much of the snapshot testing was performed using the language's data logging facility which prints the hexidecimal, ascii, and decimal values of arbitrarily-sized binary sequences. This allows straightforward investigation of any runtime values, whether that be integers, floats, pointers, strings, enums, and so on.

5.2 GenAI Comprehension Testing

With kernel development as the core case study for this project, the ability for LLMs to understand *Essence C* was assessed by tasking an LLM agent with porting a portion of a previously-written PoC operating system kernel from *C* to *Essence C*. The original kernel was developed as part of my YouTube tutorial series, and its latest source code is housed in a separate repository here. The ported version is included in this project's repository. It includes C, Essence C, NASM Assembly, and LLVM IR Assembly code living in unison, with build options to substitute different implementations of the source code.

The LLM agent was first asked to generate documentation for the language, and subsequently to port some benchmarking code into *Essence C*. The prompts and results can be

found in Appendix X.

Some slight guidance had to be given to the agent, but for the most part, it was well equipped to handle the task. This is promising as it shows that the agent took no issue with comprehending the unique language syntax and corresponding semantics (specifically, the notions of maps, symbols and modules). Note, however, that the build steps were all implemented manually as this is outside the scope of language comprehension.

5.3 Benchmarking

To kill two birds with one stone, benchmarking was performed on the actual kernel ported by the AI agent, instead of creating artificial benchmarks. Four procedures were benchmarked across three different implementations.

Procedures:

Benchmark	**Description**
PIT	Accesses the Programmable Interval Timer counter as frequently as possible via the x86 port I/O interface. This is a counter exposed by the x86 hardware that decreases rapidly over time.
RTC	Accesses the Real-Time Clock as frequently as possible via the same interface. The irony is that the RTC is furthermore used in all benchmarks to calculate the actual operations per second.
I/O Wait	Triggers the "0x80" hardware request as frequently as possible via the same interface, a legacy no-op instruction that gives the hardware a moment to breathe.
VGA Cursor	Updates the VGA text-mode cursor position as frequently as possible via the same interface. This cursor position controls where text would be printed on the screen when using the basic VGA text interface exposed by the hardware.

Table 5.1: The four kernel procedures that are benchmarked.

Implementation Variants:

5.4 Validation

Language	Mode	Details
C + NASM Assembly	N/A	Baseline implementation. Compiled with GCC with separate compilation units, with limited link-time optimization available. Instances are passed as pointer arguments, and a vtable is implemented to enable implementation swapping. This reflects a real-world C codebase that is architected to be implementation-agnostic.
Essence C + LLVM IR Assembly	Dynamic	Essence C implementation. Linked with LLVM. Instances are passed as pointer arguments, and the language uses a vtable internally to enable implementation swapping. This reflects how a programmer might architect their codebase to be implementation-agnostic.
Essence C + LLVM IR Assembly	Static	Same as above, but a custom "alwaysinline" flag is included, and a custom "mut" flag is excluded, signalling to the compiler to attempt to make instances static. The compiler succeeds without requiring the programmer to make any architectural changes to their codebase. (These optimizations could be automated at a later stage without requiring these flags).

Table 5.2: The three implementation variants tested for each benchmark.

Chapter 6 – Conclusion (930)

6.1 Overall Assessment

Given the time constraints for this project, the evaluation only scratched the surface. One would want to spend the following time:

- 200 more hours on this.
- 200 more hours on that.

Build times consistently exceeding 90 seconds. This is dreadful when debugging and making small tweaks. A professional C++ developer would likely have experience optimizing the build process.

6.2 Contribution to state-of-the-art

- inlining - built in language constructs to avoid likes of kernel static_call - continuum of dispatch, not just static vs dynamic, but by merging the notion of class instances (and fusing classes and modules), if we detect multiple instances we can switch to an even more dynamic dispatch mechanism. - more efficient inter-language interoperability

6.3 Limitations

- The prototype compiler does not yet support cross-compilation. This means a Windows executable cannot be built from within a Linux build environment, or vice versa. Thus, everything had to be kept to Linux/WSL for now.
- The parser does not produce helpful error messages. Thus, one must either use a pair of eyes to identify syntax errors, or use the bulky *Parser Trace Explorer* to investigate more significant issues.
- With more time, the entire kernel could have been migrated from *GCC* to *Clang*. This would have offered more realistic benchmarks given *Clang*'s superior link-time

optimizations over *GCC*. Nonetheless, the effort of such a migration reminds us of the struggles developers face when writing software in C today – struggles that might be less prevalent in a language like Essence C that treats LLVM IR as a first-class citizen.

6.4 Future Work

References

[1] DigiVirus. "Meet T3 chat AI: Your all-in-one, super-fast AI assistant," Accessed: Nov. 16, 2025. [Online]. Available: https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-assistant/.

[2] The LLVM Project. "The LLVM target-independent code generator," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/CodeGenerator.html.

[3] N. Sreekumar. "GSoC 2025: Introducing an ABI lowering library," Accessed: Nov. 7, 2025. [Online]. Available: https://blog.llvm.org/posts/2025-08-25-abi-library/.

[4] The LLVM Project. "LLVM's analysis and transform passes," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction.

[5] The Clang Project. "Clang: C language family frontend for LLVM," Accessed: Nov. 17, 2025. [Online]. Available: https://clang.llvm.org/.

[6] The LLVM Project. "LLVM language reference manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LangRef.html#inline-assembler-expressions.

[7] D. Roy. "GCC inline assembly and its usage in the linux kernel," Accessed: Nov. 6, 2025. [Online]. Available: https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958.

[8] The LLVM Project. "LLVM link time optimization: Design and implementation," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LinkTimeOptimization.html.

[9] The LLVM Project. "LLVM programmer's manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/ProgrammersManual.html.

[10] The LLVM Project. "Writing an LLVM pass," Accessed: Nov. 2, 2025. [Online]. Available: https://llvm.org/docs/WritingAnLLVMNewPMPass.html.

Appendix A – Record of Meetings

Maynooth University
National University of Ireland Maynooth

Department of Computer Science

RECORD OF MEETINGS FORM

Final Year Project

Record of Meetings Form: All students are required to download a copy of this form and bring it to every project meeting. You must complete the form and have it signed by the supervisor. In the case of a virtual meeting, the form should be completed and signed by your supervisor at a later stage. You are required to submit a digital copy of all meeting forms in the appendix of your final report.

Meeting Date	Student Signature	Supervisor Signature
14/10/2025		Phil Maguire
04/11/2025		Phil Maguire
18/11/2025		Phil Maguire
02/12/2025		Phil Maguire
16/12/2025		Phil Maguire
04/02/2026		Phil Maguire
18/02/2026		Phil Maguire
04/03/2026		Phil Maguire

Appendix B – LLM Comprehension Testing

B.1 Documentation Prompts

davidcalaxan:

Generate concise documentation under ai_docs/ESSENCE_C.md (one markdown file) detailing the various language features available, including code block examples of how to write valid .ec code. This must also include an explanation on how to call external LLVM IR or C code (via the C ABI only). This documentation should be designed to be efficiently used by LLM agents to port a C program into my language! Do not look at the syntax_ideas directories as these are outdated.

GitHub Copilot:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/uoe)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test2)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/uoe/ec)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/b.ec)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test2/main.ec)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#100-100), lines 100 to 316

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#100-100), lines 100 to 300

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#300-300), lines 300 to 600

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#600-600), lines 600 to 900

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#900-900), lines 900 to 1200

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1200-1200), lines 1200 to 1554

Now let me look at the backend source files for more context on types and operations:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/README.md)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_instructions.hpp)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_types.hpp#1-1), lines 1 to 150

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_hardval.hpp)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/main.cpp#1-1), lines 1 to 150

Now I have a thorough understanding of the language. Let me also check the `test/foo` directory and look at any additional examples:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/foo)

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/main.cpp#150-150), lines 150 to 200

Searched for text `sizeof|extern|ccc|alwaysinline|take_address|pointer` (`**/compiler/backend/src/main.cpp`), 13 results

Searched for text `TypeExternCcc|extern_ccc|function_name` (`**/compiler/backend/src/**`), 20 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/main.cpp#560-560), lines 560 to 764

Now I have a comprehensive understanding of the language. Let me create the documentation:

Created [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs)

Created [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md)

The documentation has been created at ESSENCE_C.md. Here's a summary of what's covered:

- **Comments** — semicolon-to-EOL syntax
- **Primitive types** — integers, floats, bool, pointer types
- **Type aliases** — `type` keyword
- **Variables** — `:=` (immutable) and `mut` + `=` (mutable), typed coercion
- **Literals** — integers, floats, strings, symbols (`:name`)
- **Enums** — inline `enum { :a; :b; }` with value selection
- **Arithmetic & comparison** — infix operators and the multi-operand **crystal** style (multi-line with leading operator)
- **Logical operators** — `&&`/`||` crystal vs pistol style, with nesting rule
- **Control flow** — `if`/`else if`/`else` and infinite `for` with `break`
- **Maps** — struct-like `{ :field value; }` literals and `:field` access
- **Callables** — `input { ... } -> { ... }` lambdas with named or positional calling
- **`sizeof` and address-of (`&`)**
- **Top-level type extensions (`@TypeName:member`)** — attaching fields and methods to types, `this` and `mod` references
- **`@Mod` module members** and the `create()` entry point
- **Extern C ABI bindings** — `extern ccc "symbol"` with tuple input/output types, named vs positional call syntax, `alwaysinline` flag, and accessing return values via `:0`
- **C-to-Essence-C mapping table** for quick porting reference
- **Full porting example** and a **module structure skeleton**

Made changes.

davidcallanan:

The only incorrect thing you included were parenthesis for nested logical statements. But parenthesis are not yet supported by the language, one needs a separate variable for now. There's also some things you can learn from the final report and the backend to make your documentation a little more comprehensive.

GitHub Copilot:

Let me read the final report and relevant backend files for more details.

Read [](file:///home/david/pigeonhole/git/mu-fyp/2603_final_report/report.tex#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/main.cpp#200-200), lines 200 to 560

Read [](file:///home/david/pigeonhole/git/mu-fyp/2603_final_report/report.tex#100-100), lines 100 to 400

Read [](file:///home/david/pigeonhole/git/mu-fyp/2603_final_report/report.tex#400-400), lines 400 to 741

Now I have a comprehensive picture. Let me also check the `evaluate_smooth` and `process_map_body` for additional type/calling info:

Searched for text `TypeExternCcc|extern_ccc|alwaysinline|take_address|sizeof` (`**/compiler/backend/src/evaluate_smooth.cpp`), 20 results

Searched for text `alwaysinline|TypeExternCcc|take_address|sizeof|mut` (`**/compiler/backend/src/process_map_body.cpp`), 2 results

Now I have everything I need. Let me rewrite the documentation with all the corrections and additions:

Using "Multi-Replace String in Files"

Let me verify the final state of the document looks correct:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#100-100), lines 100 to 260

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#260-260), lines 260 to 450

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#450-450), lines 450 to 636

The document looks correct and complete. Here's a summary of what changed:

Fixes:
- **Removed parentheses** from the nested logical operator example — parentheses are not supported. The example now uses an intermediate variable instead, with an explicit bold note: *"Parentheses are not yet supported."*
- Updated the Key Patterns table with the correct `(a && b) || c` → intermediate variable approach

Additions from the final report and backend:
- **Type-Value Unification** section — explains that types and values share the same syntax (both are constraints), so `u64`, `42`, and `u64 42` are all valid and merged by the compiler
- **Map-Enum Equivalence** section — explains how calling a map with a symbol then an argument is equivalent to a fully-populated enum variant, unifying structs, functions, and enums
- **Crystal vs Pistol Syntax** section — formally names and explains both forms for all associative operator families, and explicitly states the precedence order (multiplicative → additive → AND → OR)
- **`alwaysinline` semantics** — expanded from a vague "hint" to accurately describing that it emits `AlwaysInline` on the LLVM call function, enabling cross-module inlining via LTO, and its role in the static dispatch optimization
- **No Standard Library / Freestanding Design** section — explicitly states Essence C has no stdlib and all C functions need manual `extern ccc` declarations
- **Singletonish Optimization and `mut`** section — explains the key optimization where immutable map instances are collapsed to zero-size structs with compile-time constant fields, and how `mut` opts out of this
- **Module Structure Skeleton** — added comments explaining `input`, `this`, and `mod` in context
- Key Patterns table expanded with `const`, `volatile`/`mut`, and `alwaysinline` rows

Made changes.

davidcallanan:

Parenthesis are now supported via the (:) syntax. Please update the documentation to reflect this!

GitHub Copilot:

Let me check the current parser to understand the exact new syntax.

Searched for regex `LPAREN.*COLON|paren.*group|grouping|COLON.*RPAREN|\(:` (`**/compiler/frontend/process.js`), no results

Searched for regex `COLON|paren_group|group_expr|KW_COLON|\(:\s` (`**/compiler/frontend/process.js`), no results

Read changed files in the active git repository

Searched for regex `\(:\s|paren|PAREN_GROUP|group` (`**/compiler/frontend/process.js`), 20 results

Searched for text `LGROUP` (`**/compiler/frontend/process.js`), 4 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#175-175), lines 175 to 270

Now I have a full picture. The grouping syntax is `(: expr )` — `LGROUP` matches the regex `/^\(\s*:\s/` (i.e. `(:` optionally with spaces), and `RGROUP` is just `)`. It's used in `hardval` and `hardval_bb`, meaning it works anywhere an expression atom is valid. Let me now read the current doc and update it:

Searched for text `Parentheses are not yet supported` (`**/ai_docs/ESSENCE_C.md`), 1 result

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#160-160), lines 160 to 225

Now I have everything I need. The `(:` `)` syntax is a grouping expression usable anywhere an atom is valid. I'll update the logical operators section and add a dedicated grouping section, then fix the key patterns table:

Using "Multi-Replace String in Files"

Three changes were made to the documentation:

1. **Logical Operators section** — removed the outdated *"Parentheses are not yet supported"* note and replaced the intermediate-variable-only example with a primary `(: )` grouping example, keeping the intermediate variable form as an alternative.

2. **New "Grouping Expressions" section** — added between Logical Operators and Crystal vs Pistol Syntax, covering:
   - The `(: expr )` delimiter syntax (opening is `(:`, closing is `)`)
   - That plain `( expr )` is **not** valid for grouping
   - Multiple usage examples (arithmetic precedence, inline logical grouping, general atom context)
   - A note explaining why `:` is in the delimiter (reserved for future leaf-return semantics, keeping plain `(` unambiguous for tuple/call syntax)

3. **Key Patterns table** — updated the `(a && b) || c` row to show the `(: && a && b ) || c` inline grouping form.

Made changes.

davidcallanan:

Maps can now be mutated (via &mut or via a "mut" this context, or the mod context which is always mut). Please update the documentation to reflect this!

GitHub Copilot:

Let me examine the current state of the parser, backend, and test files to understand the exact new syntax.

Read changed files in the active git repository

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#100-100), lines 100 to 353

Searched for regex `mut.*this|KW_MUT.*ARROW|mut.*ARROW|AMPERSAND.*MUT|KW_AMPERSAND_MUT|amp_mut|&mut|map.*mut` (`**/compiler/frontend/process.js`), 2 results

Searched for text `AMPERMUT` (`**/compiler/frontend/process.js`), 3 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1280-1280), lines 1280 to 1310

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1225-1225), lines 1225 to 1255

Now let me also check map field mutation syntax and the `mut` on callable context:

Searched for regex `map_entry_assign|field.*assign|sym.*assign|SYMBOL.*EQUALS|assign.*sym` (`**/compiler/frontend/process.js`), 4 results

Searched for text `is_mutable|is_this_mutable|mutable` (`**/compiler/backend/src/t_types.hpp`), 3 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#325-325), lines 325 to 345

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_types.hpp)

Now I have a complete picture. Let me also read the current doc state around the relevant sections:

Searched for text `Address-of|Maps (Struct|Callables|Module-level|Singletonish` (`**/ai_docs/ESSENCE_C.md`), no results

Searched for text `## Address-of` (`**/ai_docs/ESSENCE_C.md`), 1 result

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#330-330), lines 330 to 430

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#430-430), lines 430 to 530

Now let me read the Key Patterns table and the Singletonish section:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#570-570), lines 570 to 678

Now I have a complete picture. Here's what's new:

1. **`&mut expr`** — mutable address-of, returns a mutable map reference
2. **`input { ... } mut -> { ... }`** — the `mut` keyword between `->` and the input type marks `this` as mutable inside the method body
3. **`this:field = value`** — field mutation using `expr_sym_assign` syntax (`ident :sym:trail = value`)
4. **`mod`** — the module context is always mutable (fields can be assigned freely)

Using "Multi-Replace String in Files"

Searched for text `mutating struct field via method` (`**/ai_docs/ESSENCE_C.md`), 1 result

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#698-698), lines 698 to 715

The `typedef` row got displaced. Let me fix that and the skeleton:

Using "Multi-Replace String in Files"

All changes are in. Here's a summary of every update made:

1. **Address-of section** — split into `&` (immutable, read-only reference) and `&mut` (mutable reference, permits field assignment), with an example of `x:name = "John Doe"` via a `&mut` binding.

2. **Callables section** — added a **"Mutable `this` context"** subsection explaining:
  - Syntax: `input { ... } mut -> { ... }` (place `mut` between the input type and `->`)
  - `this` is only writable inside a `mut` method
  - `mod` is **always** mutable — no `mut` needed to assign module-level fields

3. **New "Map Field Mutation" section** — after `sizeof`, covering all three mutable contexts with examples: `&mut` reference, `mut` callable `this`, and `mod` fields; plus a note that assignment through immutable `&` or non-`mut` `this` is a compile error.

4. **Singletonish Optimization section** — added a paragraph clarifying that `&mut` prevents the singletonish optimization (because fields may be mutated), whereas `&` preserves it.

5. **Key Patterns table** — added rows for mutable ref (`&mut`), and mutating struct field via method; restored `typedef` row that was previously displaced.

6. **Module Structure Skeleton** — updated `@MyType:field_name` to have an initial value (`u64 0`), added a `@MyType:set_field` `mut` method example, and noted that `mod` is always mutable.

Made changes.

davidcallanan:

Please update the documentation to reflect how callables can now return data via a named type.

GitHub Copilot:

Let me examine the current state of the relevant code.

Searched for text `call_output_type_named` (`**/compiler/frontend/process.js`), 3 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#80-80), lines 80 to 353

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1285-1285), lines 1285 to 1320

Now I understand: when the output type is `TypeName { ... }` (a `type_constrained` whose first constraint is a named type), the callable's output is identified as a named return type. Looking at the test file:

```ec
```

```
type Return (u64);

@Mod:new_function () -> Return {
   :0 12;
};
```

The callable body `Return { :0 12; }` is `type_constrained` — the output is a named type (`Return`) followed immediately by a map body. Let me check how the body is separated from the named type in the grammar:

Searched for text `call_output_type_named` (`**/compiler/backend/src/normalize_type.cpp`), 3 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/normalize_type.cpp#200-200), lines 200 to 270

Now I have a complete picture. The pattern is:

```ec
type Return (u64);          ; define a named return type (tuple alias)

@Mod:fn () -> Return {      ; Return is the named type, { ... } is the body
   :0 12;
};

mod:fn():0                  ; access positional return value
```

The named type comes right before the body braces — `() -> ReturnType { body }`. The return type must itself be a map type (e.g. a tuple alias like `type Return (u64)`). Let me now read the current doc sections to update:

Searched for text `## Callables|## Accessing Return Values|return value is the last` (`**/ai_docs/ESSENCE_C.md`), 3 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#311-311), lines 311 to 400

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#585-585), lines 585 to 615

Now I have everything needed. I'll update the Callables section to add a named return type subsection, and expand the Accessing Return Values section to explain named return types clearly.

Using "Multi-Replace String in Files"

Two sections updated:

1. **Callables — new "Named return type" subsection** — explains:
   - Syntax: place a type name between `->` and the body braces (`() -> Result { :0 42; }`)
   - The named type must be a map type, typically a tuple alias (`type Result (u64);`)

- Multi-value example with `type Vec2 (f32, f32)`
- Contrast with anonymous return (`() -> (i32) { ... }`) where the shape is inferred
- Forward reference to the Accessing Return Values section

2. **Accessing Return Values — expanded** — now explains:
   - Positional access via `:0`, `:1`, etc. applies to all callables and `extern ccc` functions
   - Inline chaining syntax (`mod:getpid():0`)
   - How named return types shape the result map, with a complete round-trip example showing both the declaration and the call-site access

Made changes.

davidcallanan:

You made a mistake - anonymous return types are not inferred at the moment.

GitHub Copilot:

Searched for text `anonymous tuple output` (`**/ai_docs/ESSENCE_C.md`), 1 result

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#380-380), lines 380 to 415

Using "Replace String in File"

Replaced the incorrect anonymous inference paragraph with a clear note that a named return type is **required** to return data, and that `-> { }` (empty body map) is the correct form for void callables.

Made changes.

davidcondensis:

Please update the documentation to reflect the new operators available. Please also include how callables can return a map using field names instead of positional syntax in, for example, factories.

GitHub Copilot:

Summarized conversation history

Let me examine the current state of the test file and parser to understand the new operators and map return syntax.

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec)

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#1-1), lines 1 to 100

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#100-100), lines 100 to 300

Searched for text `b&|<<|>>|bitwis|shift|modulo|TypeExprNot|TypeExprShiftLeft|TypeExprBitwise` (`**/compiler/frontend/process.js`), 20 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#460-460), lines 460 to 620

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#300-300), lines 300 to 813

Now let me look at the parser more carefully for the new operators, their tokens, and check for any map-return factory syntax:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#95-95), lines 95 to 130

Searched for text `call_output_type_named|output_fields|return.*map|map.*return|factory|named.*field|field.*return` (`**/compiler/frontend/process.js`), 3 results

Searched for text `output_fields|named_fields|return_named|:name|field_return` (`**/compiler/frontend/process.js`), no results

Searched for text `TypeSymAssign|sym_assign|field_assign|return_sym|named_return` (`**/compiler/frontend/process.js`), 4 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#620-620), lines 620 to 680

Searched for text `named.*field.*return|field.*name.*result|sym.*return|return.*sym|map_return` (`**/compiler/frontend/process.js`), no results

Now let me look at how the named return type body works and how fields can be set by name (not just `:0`):

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1270-1270), lines 1270 to 1350

Searched for text `bitwise_and|bitwise_or|shift_left|shift_right|modulo|expr_not` (`**/compiler/backend/src`), no results

Searched for text `bitwise_and|shift_left|shift_right|expr_modulo|expr_not` (`**/compiler/backend/src/main.cpp`), no results

Searched for text `bitwise_and|shift_left|shift_right|modulo|expr_not` (`**/compiler/backend/src/**`), 20 results

Now I have a good picture of all the new operators. Let me also check the `!` (NOT) operator in the parser and the `b|` (bitwise OR) token, and look at the test file's `w` / `w2` / `w3` / `w4` section again more closely to confirm the exact syntax used:

The test file clearly shows:
```ec
w := 23 b& 41;
w2 := (: u64 19) << 2;
w3 := bool :true;
w4 := !w3;
```

And from parser tokens: `b&`, `b|`, `<<`, `>>`, `%`, `!`. Now let me look at how the named return type can use field names (not just positional `:0`, `:1`):

Searched for text `:name|:x|:y|named.*return|return.*named|factory` (`**/compiler/test/**`), 20 results

Searched for text `call_output_type_named|sym_name|field_name|named_field` (`**/compiler/backend/src/normalize_type.cpp`), 6 results

Now let me look at the `type Return` declaration — since the named return type uses a `type` alias, the body field names come from the alias. Let me check how named sym fields work in type definitions and whether a named return type with named fields (not just positional) is already supported:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/normalize_type.cpp#200-200), lines 200 to 260

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_types.hpp#140-140), lines 140 to 170

Now let me understand the named return with named fields. When a `type` is declared with a map body (not just `(u64)` tuple), the fields are named. Let me check what the frontend would produce for a type with named map fields and how the body would use them:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_types.hpp#1-1), lines 1 to 140

Now I have a comprehensive understanding. The named return type with **named fields** (not just positional `:0`) is how factories work — the `type` declaration establishes the sym fields, and the callable body sets them by name. Let me verify this understanding by checking the `type_named` handling and whether field names can be set by symbol in the return body:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/normalize_type.cpp#880-880), lines 880 to 920

The `extern ccc` return type must use `:0` only. But for regular Essence C callables with named return types, the body can use named sym fields since the body is just a map. Let me look at the `type` keyword with a map body (like `type Vec2 { :x f32; :y f32; }`) to verify factory-style named field setting:

Searched for text `type_named|KW_TYPE|top_type|type_alias` (`**/compiler/frontend/process.js`), 20 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1356-1356), lines 1356 to 1380

Good. Now I have all I need. The `type` declaration can define any map, and the named return type uses the map's field names. When you write `type Person { :name *u8; :age u64; }` and `() -> Person { :name "Alice"; :age 30; }`, the body sets named fields. Let me check the 2511_syntax_ideas files for any factory examples:

Searched for files matching `**/compiler/test2/**`, 1 match

Searched for text `factory|Person|struct.*return|return.*struct|named.*field.*return` (`**/compiler/test/**`), no results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test2/main.ec)

Now I have everything I need. Let me now update the documentation:

1. **New operators**: `%` (modulo, pistol-only), `<<` / `>>` (shifts, pistol-only), `b&` / `b|` (bitwise AND/OR, both crystal and pistol), `!` (logical NOT, unary prefix)
2. **Named-field return**: when the named return type is a map type with named sym fields (not just a tuple), the body sets fields by name (`:fieldname value;`)

Let me read the exact current Arithmetic section in the doc:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#140-140), lines 140 to 240

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#240-240), lines 240 to 260

Now let me look at the Named return type section and Accessing Return Values section to know exactly what to expand:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#350-350), lines 350 to 450

Now I have all the context I need. Let me make all the updates:

1. **Arithmetic & Comparison** — add modulo `%` and shifts `<<` / `>>`
2. **Bitwise Operators** — new section for `b&` / `b|` (crystal + pistol)
3. **Logical NOT** — add `!` as unary prefix
4. **Crystal vs Pistol** — update precedence chain to include new ops
5. **Named return type** — add named-field factory example
6. **Key Patterns table** — add new operators

Using "Multi-Replace String in Files"

Now let me verify the result looks correct around the key sections:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#130-130), lines 130 to 280

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#370-370), lines 370 to 480

Everything looks correct. Let me quickly verify the updated Key Patterns table and Crystal vs Pistol section:

Searched for text `bitwise NOT|b&|b\\||shift|modulo|%|Precedence order` (`**/ai_docs/ESSENCE_C.md`), 20 results

Searched for regex `Precedence order|b&|b\||shift|modulo|Key Patterns` (`**/ai_docs/ESSENCE_C.md`), 20 results

All changes are in place. Here's a summary of what was updated in ESSENCE_C.md:

New content added

Arithmetic & Comparison section — extended with:
- `%` modulo (pistol-only binary infix)
- `<<` / `>>` bit shifts (pistol-only; note that a type-coercion left operand requires `(: )` grouping)

New "Bitwise Operators" section — inserted between Arithmetic and Logical Operators:
- `b&` / `b|` — both support crystal (multi-operand, leading-operator-per-line) and pistol (binary infix) forms, exactly mirroring `&&` / `||`
- `!` — logical NOT, unary prefix on `bool` values

Crystal vs Pistol section — updated:
- Added a bitwise crystal example in the code block
- Added a note that bitwise NOT does not exist — use `!` on `bool` only
- Updated the precedence chain: `multiplicative → additive → modulo → shifts → comparison → bitwise AND → bitwise OR → logical AND → logical OR`

Named return type section — restructured into two subsections:
- **Positional (tuple-style) return** — existing `:0`, `:1` pattern
- **Named fields (factory-style) return** — new: when the return `type` is declared with `{}` and `@Type:field` extensions, the callable body populates fields by name (`:name "Alice";`), and the caller accesses them the same way (`p:name`)

Key Patterns table — added rows for `!a`, `a & b` → `b&`, `a | b` → `b|`, `<<`, `>>`, `%`.

Made changes.

davidcallanan:

The language now supports nullptr, *void and &MyType reference casts. Can you update the documentation to reflect this?

GitHub Copilot:

Optimizing tool selection...

Let me explore the current state of the parser and test files to understand the exact syntax for these new features.

Searched for text `nullptr|void|cast|reference.*cast|NULLPTR|VOID`, 20 results

Searched for text `nullptr|*void|cast|&MyType|reference`, no results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#325-325), lines 325 to 353

Searched for regex `nullptr|voidptr|void_map|KW_NULLPTR|type_nullptr|type_voidptr|void_map_reference|VOID|void`, 20 results

Searched for regex `KW_NULL|nullptr|voidptr|\*void|void_map|type_nullptr|type_voidptr|type_void_map`, 20 results

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/test/a.ec#353-353), lines 353 to 362

Now let me look at the parser rules for `type_void_map_reference` and how `&MyType ptr` is parsed:

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#180-180), lines 180 to 280

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1230-1230), lines 1230 to 1280

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/frontend/process.js#1280-1280), lines 1280 to 1360

Read [](file:///home/david/pigeonhole/git/mu-fyp/compiler/backend/src/t_types.hpp#195-195), lines 195 to 209

Now let me look at the evaluate_smooth.cpp or `better_leaf_agnostically_translate.cpp` to understand what `TypeVoidMapReference` does — is it a reinterpret/bitcast of a pointer to a typed reference?

Searched for text `TypeVoidMapReference|void_map_reference|VoidMapReference`, 20 results

Searched for text `TypeVoidMapReference|void_map_reference|VoidMapReference|voidptr|TypeVoidptr|TypeNullptr`, 20 results

Now I have a clear picture of all three features. Let me also check the benchmark `.ec` file to see `*void` used as a type in parameters:

Read [](file:///home/david/pigeonhole/git/mu-fyp/benchmarks/kernel/source/src/modern/port_manager.ec#95-95), lines 95 to 103

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#455-455), lines 455 to 490

Now I have everything I need. Let me understand the semantics clearly from the code:

1. **`nullptr`** — keyword, evaluates to a `*void` null pointer value. Its underlying type is `TypeVoidptr` (null `*void`).
2. **`*void`** — type keyword for an untyped pointer (`void*` in C). Used as a type annotation or as a constraint (e.g. `*void nullptr` coerces `nullptr` to explicit `*void` type).
3. **`&MyType ptr` / `&mut MyType ptr`** — reference cast: takes an existing `*void` pointer value and reinterprets it as a typed map reference (`&MyType` or `&mut MyType`). The parser matches `&TypeName hardval_bb` or `&TypeName constraint_map_bb` as a `type_void_map_reference` constrained with the value. The `target` is the named type.

Now let me find the right place in the doc to insert a new section and update the Primitive Types table and Key Patterns table:

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#15-15), lines 15 to 45

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#490-490), lines 490 to 560

Now I have all the context needed. I'll make three focused changes:

1. Add `*void` and `nullptr` to the Primitive Types table
2. Add a new **Pointers, Null, and Reference Casts** section after the Address-of section
3. Update the Key Patterns table

Using "Multi-Replace String in Files"

Let me verify the new section looks correct:

Searched for regex `nullptr|Pointers, Null|reference cast|Reference Cast|\*void`, 20 results

Everything looks correct. Here's a summary of the three changes made to ESSENCE_C.md:

Changes made

Primitive Types table — added two new rows:
- `*void` — untyped pointer (`void*` in C)
- `nullptr` — null pointer literal (type `*void`, zero address)

New section: "Pointers, Null, and Reference Casts" — inserted after the Address-of section, with three subsections:

- **`nullptr`** — keyword that evaluates to a null `*void` pointer. `*void nullptr` gives it an explicit type annotation; bare `nullptr` works where the type is resolved by context (e.g. passing to an `extern ccc` function).
- **`*void`** — the untyped pointer type, equivalent to C's `void*`. Used in `extern ccc` bindings for functions like `malloc`/`free`. Fields cannot be accessed through a `*void` — a reference cast is required first.
- **Reference casts (`&TypeName ptr` / `&mut TypeName ptr`)** — reinterprets a `*void` pointer as a typed map reference. No runtime conversion — purely a type-level reinterpretation (C pointer cast equivalent). Mutability rules mirror ordinary address-of: `&TypeName` is read-only, `&mut TypeName` allows field assignment.

Key Patterns table — added rows for:
- `void* p = NULL;` → `p := *void nullptr;`
- `void* p = malloc(n);` → `raw := mod:malloc(n); p := raw:0;`
- `(MyType*)ptr` immutable → `&MyType ptr`
- `(MyType*)ptr` mutable → `&mut MyType ptr`

Made changes.

davidcallanan:

The cast doesn't work "inline with a map literal".

GitHub Copilot:

Searched for text `inline with a map literal`, 1 result

Read [](file:///home/david/pigeonhole/git/mu-fyp/ai_docs/ESSENCE_C.md#600-600), lines 600 to 618

Using "Replace String in File"

Done — removed the incorrect inline map literal example.

Made changes.

B.2 Generated Documentation PDF

Essence C Language Reference

Essence C (`.ec`) is a compiled, statically-typed language that compiles to native code via LLVM. This document is intended as a concise reference for porting C programs into Essence C.

Comments

Comments begin with `;` and run to end of line. There are no block comments.

```
; this is a comment
x := 10; this is also a comment
```

Primitive Types

Type	Description
`u8`, `u16`, `u32`, `u64`	Unsigned integers
`i8`, `i16`, `i32`, `i64`	Signed integers
`usize`	Pointer-sized unsigned integer
`f32`, `f64`	Floating-point
`bool`	Boolean
`*T`	Pointer to `T`
`**T`	Pointer to pointer to `T`
`*void`	Untyped pointer (`void*` in C)
`nullptr`	Null pointer literal (type `*void`, zero address)

Type Aliases

Use `type` at the top level to define an alias for an existing type.

```
type MyInt u64;
type String *u8;
type CharPtrPtr **u8;
type MyStruct {};
```

An empty `{}` defines an opaque map type (struct-like).

Variables

Declare with `:=` (immutable by default). Use `mut` for mutable variables.

```
x := 42;
name := "hello";
ptr := *u8 "hello";
mut counter := u64 0;
```

Assignment to a mutable variable uses `=`:

```
counter = counter + 1;
```

Typed coercion syntax — `TypeName value` — coerces a literal or variable to a specific type:

```
n := u64 10;
f := f32 3.14;
s := String "world";
flag := bool :true;
```

Literals

- **Integer**: `42`, `0`, `1000`
- **Float**: `3.14`, `0.5`
- **String**: `"hello\n"` — supports `\n` and `\\` escape sequences
- **Symbol (enum variant)**: `:true`, `:false`, `:ok`, `:error`

Symbols and Enums

Symbols are prefixed with `:`. An enum type is declared inline with `enum { }`:

```
flag := bool :true;

status := enum {
    :ok;
    :error;
} :ok;
```

Single-line form:

```
status := enum { :ok, :error } :ok;
```

Type-Value Unification

In Essence C, types and values share the same syntax — everything is a *constraint*. A literal value is simply a singleton constraint (exactly one possible value). The compiler merges multiple constraints to determine the actual runtime representation.

This means a variable can be assigned to:

- A pure type (a non-singleton constraint, e.g. just `u64`)
- A pure value (a singleton constraint, e.g. just `42`)
- Both together (a type coercion, e.g. `u64 42`)

```
a := u64;       ; type only — no runtime value yet (used in struct field declarations)
b := 42;        ; value only — compiler infers minimal integer type
c := u64 42;    ; type + value — explicit coercion
```

The compiler decides the encoding (bit-width, representation) by merging all constraints it can observe.

Map-Enum Equivalence

Calling a map with a symbol (`:foo`) and then calling the result with another argument is semantically identical to calling the map with a fully populated enum variant (`:foo(arg)`). This is the *map-enum equivalence* rule, which unifies structs, functions, and enums into one coherent concept.

In practice this means you rarely need to think about enums vs structs — both are just maps accessed via symbols.

Arithmetic & Comparison

Standard infix operators for binary expressions:

```
result := a + b;
result := a - b;
result := a * b;
result := a / b;
```

Multi-operand **additive crystal** (all operands at same precedence, each on its own line):

```
total :=
    + 50
    + 70
    - 30
;
```

Modulo:

```
remainder := n % 4;
```

Bit shifts (right-hand operand is the shift amount):

```
left  := value << 3;
right := value >> 2;
; grouping is required when the left operand is a type-coercion expression
shifted := (: u64 19) << 2;
```

Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`.

```
is_ten := x == 10;
is_positive := x > 0;
```

Bitwise Operators

Bitwise AND uses the token `b&`; bitwise OR uses `b|`. Both support crystal (multi-operand) and pistol (binary infix) forms, matching the same pattern as logical AND/OR.

```
; pistol (binary infix)
mask  := flags b& 0xFF;
combo := a b| b;

; crystal (multi-operand)
result :=
    b& value_a
    b& value_b
    b& 0xF0
;

or_result :=
    b| channel_a
    b| channel_b
;
```

Logical NOT is a unary prefix operator `!` applied to a `bool` value:

```
flag   := bool :true;
neg    := !flag;      ; bool :false
```

Logical Operators

Logical AND and OR use prefix-style multi-operand crystal syntax:

```
both :=
    && condition_a
    && condition_b
;

either :=
    || condition_a
    || condition_b
;
```

Do not mix `&&` and `||` in one sequence; nest them using the grouping syntax `(: expr )` (see below) or an intermediate variable:

```
; using (: ) grouping
result :=
    || (: condition_a && condition_b )
    || condition_c
;

; equivalently, using an intermediate variable
both :=
    && condition_a
    && condition_b
;

result :=
    || both
    || condition_c
;
```

Grouping Expressions

Wrap any expression in `(:` and `)` to form a grouped sub-expression. The opening delimiter is `(:` (optionally with whitespace before the inner expression); the closing delimiter is `)`. This is the only parenthesis-grouping syntax in the language — plain `( expr )` is **not** valid for grouping.

```
; force evaluation order in arithmetic
x := (: a + b ) * c;

; group a logical sub-expression inline
result :=
    || (: && condition_a && condition_b )
    || condition_c
;

; works anywhere an expression atom is valid
flag := (: bool :true );
```

The `:` in the delimiter is intentional: it is reserved for a future "leaf return" feature, so plain `(` remains available for existing tuple/call syntax without ambiguity.

Crystal vs Pistol Syntax

All associative operator families (additive, multiplicative, logical AND, logical OR) support two equivalent syntactic forms:

- **Crystal** — a unary-style batch where each operand is prefixed by its operator. This extends naturally to any number of operands and reads top-to-bottom.
- **Pistol** — a conventional left-to-right binary infix form.

These forms are interchangeable; precedence is the same. The crystal form is preferred for multi-operand expressions.

```
; Crystal (multi-operand, leading operator per line)
total :=
    + a
    + b
    - c
;

; Pistol (binary infix)
total := a + b - c;

; Both are legal for multiplication too
product :=
    * x
    * y
;

; Bitwise AND and OR follow the same crystal/pistol pattern
mask :=
    b& value_a
    b& value_b
;

; Bitwise NOT does NOT exist — use logical NOT (!) on bool values only
```

Precedence order (highest to lowest): multiplicative → additive → modulo → shifts → comparison → bitwise AND → bitwise OR → logical AND → logical OR.

Control Flow

`if` / `else if` / `else`

```
if (x == 0) {
    log("zero");
} else if (x < 0) {
    log("negative");
} else {
    log("positive");
}
```

`for` (infinite loop)

```
for {
    ; ... body ...
    if (done) {
        break;
    }
}
```

Only `break` is available to exit a loop. There is no `continue`.

Maps (Struct-like Objects)

A map literal is `{ :sym value; ... }`. Members are accessed with `:sym` chained after the variable.

```
person := {
    :name "Alice";
    :age u64 30;
};

log(person:name);
n := person:age;
```

Nested maps:

```
data := {
    :inner {
        :x u64 1;
    };
};

v := data:inner:x;
```

Callables (Functions / Lambdas)

A callable is declared with `input { ... } -> { ... }`. `input` is optional as a label but required syntactically.

```
greet := input {
    :name *u8;
} -> {
    log(input:name);
};

greet {
    :name "Bob";
};
```

Callables can also be written inline inside a map:

```
obj := {
    input {
        :x u64;
    } -> {
        log_d(input:x);
    }
};
```

Tuple-style calling (positional arguments)

Instead of named map arguments you can use parentheses with positional values. Positional parameters are accessed via `:0`, `:1`, etc.:

```
greet("Alice");          ; calls with :0 = "Alice"
add(10, 20);             ; calls with :0 = 10, :1 = 20
```

Mutable `this` context (`mut` on callable)

By default, `this` inside a method body is immutable. Place `mut` between the input type and `->` to mark `this` as mutable, allowing field assignment inside the method:

```
@MyService:set_name input {
    :new_name *u8;
} mut -> {
    ; "this" is mutable here
    this:name = input:new_name;
};
```

The `mod` context is **always** mutable — module-level fields can be assigned from any method without needing `mut`.

A method that does not declare `mut` cannot modify `this` fields (the compiler will reject the attempt).

Named return type

A callable can declare a named return type by placing a type name between `->` and the body braces. The named type must itself be a map type declared with `type`. The body then populates that type's fields.

A named return type is **required** to return data — anonymous return type inference is not currently supported. Use `-> { }` (empty body map) for callables that return nothing:

```
@Mod:greet () -> {
    log("hello");
};
```

Positional (tuple-style) return

Declare the return type as a tuple using parenthesised types. Fields are accessed via `:0`, `:1`, etc.:

```
type Result (u64);   ; single positional field

@Mod:compute () -> Result {
    :0 42;
};

type Vec2 (f32, f32);

@Mod:origin () -> Vec2 {
    :0 f32 0.0;
    :1 f32 0.0;
};
```

Named fields (factory-style) return

When the return type is a map type with named sym fields (declared with `{}`), the callable body sets fields by name instead of position. This is the idiomatic pattern for factories:

```
type Person {};

@Person:name *u8;
@Person:age  u64;

; factory: returns a fully-initialised Person by field name
@Mod:make_person input {
    :name *u8;
    :age  u64;
} -> Person {
    :name input:name;
    :age  input:age;
};

create() -> {
    p := mod:make_person {
        :name "Alice";
        :age  u64 30;
    };
    log(p:name);
}
```

The returned map inherits the type's sym layout, so callers access fields via `:name`, `:age`, etc. rather than `:0`, `:1`.

Access positional return values via `:0`, `:1`, etc. on the call result (see **Accessing Return Values**).

`sizeof`

Returns the size in bytes of a value or type expression.

```
s := sizeof(x);
log_d(s);
```

Map Field Mutation

Fields of a map can be assigned using `variable:sym:... = value`. This is only legal when the map reference is mutable — obtained via `&mut`, a `mut` callable `this`, or the `mod` context.

```
; via &mut reference
x := &mut {
    :count u64 0;
    :label "initial";
};

x:count = x:count + 1;
x:label = "updated";

; nested field
x:inner:value = 42;
```

```
; via mut this inside a method
@Counter:increment input {} mut -> {
    this:count = this:count + 1;
};
```

```
; mod fields are always mutable
@Mod:count u64 0;

@Mod:tick input {} -> {
    mod:count = mod:count + 1;
};
```

Attempting to assign a field through an immutable `&` reference or a non-`mut` `this` is a compile error.

Address-of

`&expr` takes an immutable address of an expression (returns a read-only map reference / pointer):

```
ptr := &some_value;
```

`&mut expr` takes a **mutable** address, returning a mutable map reference whose fields can be assigned:

```
x := &mut {
    :name "David";
};

x:name = "John Doe";   ; field mutation via mutable reference
```

Only `&mut` references permit field assignment. `&` references are read-only.

Pointers, Null, and Reference Casts

`nullptr` — null pointer literal

`nullptr` is a keyword that evaluates to a null pointer. Its type is `*void` (an untyped pointer). Use `*void` as an explicit type annotation when passing null to functions that expect an untyped pointer:

```
ptr := *void nullptr;   ; typed null *void pointer
```

`nullptr` can also appear without an explicit type annotation where the surrounding context resolves the type:

```
; as an argument to an extern ccc function that expects *void
mod:some_func(nullptr);
```

`*void` — untyped pointer type

`*void` is the Essence C equivalent of C's `void*`. It is used as a type in `extern ccc` bindings that deal with untyped memory (e.g. `malloc`, `free`, `memcpy`):

```
@Mod:malloc extern ccc "malloc" (usize) -> (*void);
@Mod:free   extern ccc "free"   (*void) -> ();

create() -> {
    raw := mod:malloc(u64 1024);
    mod:free(raw:0);
}
```

Note that `*void` is a distinct type from typed pointers such as `*u8`. You cannot directly read fields through a `*void` pointer — you must first cast it to a typed reference.

Reference casts — `&TypeName ptr` and `&mut TypeName ptr`

A `*void` pointer can be reinterpreted as a typed map reference using a **reference cast**. The syntax places `&TypeName` (or `&mut TypeName`) before the pointer value:

```
; immutable reference cast
obj := &MyService ptr;

; mutable reference cast
obj_mut := &mut MyService ptr;
```

Where `ptr` is a `*void` value (for example, the return value of `malloc` or a pointer received from C). The cast tells the compiler to treat the raw pointer as a reference to a `MyService` map. No runtime conversion occurs — it is a purely type-level reinterpretation (equivalent to a C pointer cast).

Mutability follows the same rules as ordinary references: `&TypeName` produces an immutable reference (read-only field access); `&mut TypeName` produces a mutable reference (field assignment permitted).

Top-level Type Extensions (`@`)

Attach named members (including callables and extern functions) to a named type using `@TypeName:member_name value`:

```
type Vec2 {};

@Vec2:x f32;
@Vec2:y f32;

@Vec2:length input {} -> {
    ; method body; "this" refers to the Vec2 instance
    log(this:x);
};
```

Multi-level paths for nested member attachment:

```
@Vec2:math:dot input {
    :other Vec2;
} -> {
    ; ...
};
```

`this` refers to the receiver object. `mod` refers to the current module.

Module-level Members (`@Mod`)

Attach members to the current module using `@Mod:name value`:

```
@Mod:app_name "My App";

@Mod:run input {} -> {
    log(mod:app_name);
};
```

Call module members via `mod:name`:

```
mod:run {};
```

Entry Point

Every module must have exactly one `create` block. This is the entry point. By default the compiled entry function is exported as `main`:

```
create() -> {
    ; program starts here
}
```

`extern ccc` on `create` — custom export symbol

Use `extern ccc "symbol_name"` on the `create` block to export the entry point under a custom C-ABI symbol name instead of `main`. This is useful when the module is intended to be called from C or another language (e.g. a kernel entry point, a library initialiser, or a custom linker script symbol):

```
create extern ccc "kernel_main" () -> {
    ; exported as "kernel_main" via the C calling convention
}
```

Without `extern ccc` the function is still exported as `main` and callable from C, but the `extern ccc` form makes the symbol name explicit.

Calling External C / LLVM IR Functions (C ABI)

Use `extern ccc "symbol_name"` inside an `@` extension to bind a C ABI function. `ccc` means the C Calling Convention.

Syntax

```
@TypeName:method_name extern ccc "c_function_name" (param_types) -> (return_types);
```

Parameters and return types use tuple syntax. An empty tuple `()` means void.

Examples

```
type Libc {};

; void printf(const char* fmt, ...)
; Note: variadic args not yet supported; bind single-call signatures
@Libc:printf extern ccc "printf" (*u8) -> ();

; int getpid(void)
@Libc:getpid extern ccc "getpid" () -> (i32);

; void* malloc(size_t n)
@Libc:malloc extern ccc "malloc" (u64) -> (*u8);

; void free(void* ptr)
@Libc:free extern ccc "free" (*u8) -> ();

; int puts(const char* s)
@Libc:puts extern ccc "puts" (*u8) -> (i32);
```

Calling extern functions

Attach to `Mod` to call without instantiating a type object:

```
@Mod:printf extern ccc "printf" (*u8) -> ();
@Mod:getpid extern ccc "getpid" () -> (i32);

create() -> {
    mod:printf("Hello, World!\n");

    result := mod:getpid();
    log_d(result:0);     ; access first return value with :0
}
```

Named argument form (using `:0`, `:1` for positional):

```
mod:printf {
    :0 "Hello\n";
};
```

`alwaysinline` call flag

Apply `alwaysinline` between the callable and its argument map to instruct the LLVM backend to emit the `alwaysinline` attribute on the generated call function, forcing the optimizer to inline the call across the call boundary. This is the key flag for the *static* implementation variant, enabling cross-module inlining via LTO.

```
mod:printf alwaysinline {
    :0 "Inlined!\n";
};
```

The `alwaysinline` flag can also be used with named-argument map syntax:

```
some_callable alwaysinline {
    :x u64 10;
};
```

Accessing Return Values

Return values (from any callable or `extern ccc` function) are accessed positionally via `:0`, `:1`, etc. on the call result map:

```
ret := mod:getpid();
pid := ret:0;
log_d(pid);

; or inline, chaining the access directly on the call expression
log_d(mod:getpid():0);
```

When a callable uses a **named return type**, the result map is shaped by that type. Fields are still accessed positionally (or by name if the type defines named sym fields):

```
type Result (u64);

@Mod:compute () -> Result {
    :0 42;
};

create() -> {
    res := mod:compute();
    log_d(res:0);              ; 42

    ; or inline
    log_d(mod:compute():0);
}

---

## Debugging Utilities

These are built-in and do not require extern declarations.

```ec
log("some string");        ; print a string
log();                     ; print empty line
log_d(value);              ; print a value as hex + ascii + decimal
log_dd(ptr, null-term);    ; dereference and print a null-terminated string
log_dd(ptr, 16);           ; dereference and print 16 bytes
```
```

No Standard Library — Freestanding Design

Essence C has no standard library. It is designed for freestanding environments (e.g. OS kernels). The only built-in external dependency is `puts` (used internally by `log` / `log_d`). All other library functions must be declared as `extern ccc` bindings manually.

This means all C standard library functions (`printf`, `malloc`, `free`, `memcpy`, etc.) must each have an `@Mod:name extern ccc "symbol" (...) -> (...);` declaration before use.

Singletonish Optimization and `mut`

By default, variables are immutable (`const`). When the compiler can prove that a map instance holds only compile-time-constant data (no mutable state), it applies the *singletonish* optimization: the map is collapsed to a zero-size struct and all its field accesses become compile-time constants, allowing the optimizer to eliminate runtime overhead entirely.

Declaring a variable with `mut` opts out of this optimization for that variable, signalling that the value may change at runtime:

```
mut counter := u64 0;   ; runtime-mutable; singletonish NOT applicable
counter = counter + 1;
```

Similarly, taking a **mutable address** (`&mut`) of a map prevents the singletonish optimization on that map, because the map's fields may be mutated through the reference. Taking an immutable address (`&`) preserves the optimization opportunity:

```
x := &some_map;      ; immutable reference — singletonish may still apply
y := &mut some_map;  ; mutable reference — singletonish does NOT apply
```

For the *static dispatch* variant of a module, omitting `mut` on instance variables (combined with `alwaysinline` calls) is what allows the compiler to fully inline and eliminate indirection.

Full Example: Porting a C Program

Original C:

```
#include <stdio.h>
#include <stdlib.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    int x = add(3, 4);
    printf("result: %d\n", x);
    return 0;
}
```

Essence C equivalent:

```
type Libc {};

@Libc:printf extern ccc "printf" (*u8) -> ();

@Mod:printf extern ccc "printf" (*u8) -> ();

add := input {
    :a i32;
    :b i32;
} -> {
    ; return value is the last expression in the block
    input:a + input:b
};

create() -> {
    result := add {
        :a i32 3;
        :b i32 4;
    };

    mod:printf("result: %d\n");   ; NOTE: formatted printing requires the real printf with variadic args
                                   ; for now, use log_d for numeric output
    log_d(result);
}
```

Key Patterns When Porting C

C concept	Essence C equivalent
`int x = 5;`	`x := i32 5;`
`int* p = &x;`	`p := &x;` (immutable ref)
`int* p = &x; *p = 5;`	`p := &mut x; p:field = 5;` (mutable ref)
`void* p = NULL;`	`p := *void nullptr;`
`void* p = malloc(n);`	`raw := mod:malloc(n); p := raw:0;`

C concept	Essence C equivalent
`(MyType*)ptr`	`&MyType ptr` (immutable ref cast from `*void`)
`(MyType*)ptr` (mutable)	`&mut MyType ptr` (mutable ref cast from `*void`)
mutating struct field via method	`input { ... } mut -> { this:field = val; }`
`typedef int MyInt;`	`type MyInt i32;`
`struct Foo { int x; }`	`type Foo {};` then `@Foo:x i32;`
`void fn(int a)`	`@Mod:fn input { :a i32; } -> { ... };`
`extern int puts(const char*)`	`@Mod:puts extern ccc "puts" (*u8) -> (i32);`
`puts("hi")`	`mod:puts("hi");`
custom entry symbol (e.g. `_start`)	`create extern ccc "_start" () -> { ... }`
`for (;;) { ... break; }`	`for { ... break; }`
`if (c) { } else { }`	`if (c) { } else { }`
`a && b`	`&& a && b` (crystal) or `a && b` (pistol)
`a || b`	`|| a || b` (crystal) or `a || b` (pistol)
`(a && b) || c`	`|| (: && a && b ) || c` or use intermediate variable
`!a`	`!a` (unary prefix, `bool` only)
`a & b` (bitwise AND)	`a b& b` (pistol) or `b& a b& b` (crystal)
`a | b` (bitwise OR)	`a b| b` (pistol) or `b| a b| b` (crystal)
`a << n`	`a << n`
`a >> n`	`a >> n`
`a % n`	`a % n`
`sizeof(x)`	`sizeof(x)`
`__attribute__((always_inline))`	`alwaysinline` call flag
`const int x = 5;`	`x := i32 5;` (immutable by default)
`volatile int x = 5;`	`mut x := i32 5;`

Module Structure Skeleton

```
; 1. Type aliases
type MyType {};

; 2. Type extensions (fields and methods)
; Inside a method body:
;   "input" refers to the call argument map
;   "this"  refers to the receiver instance (the MyType object)
;   "mod"   refers to the current module (Mod) — always mutable
@MyType:field_name u64 0;
@MyType:do_thing input { :x u64; } -> {
    log_d(input:x);
    log(this:field_name);   ; access receiver field
};
@MyType:set_field input { :x u64; } mut -> {
    this:field_name = input:x;  ; mutation requires "mut" on the callable
};
@MyType:set_field input { :x u64; } mut -> {
    this:field_name = input:x;  ; mutation requires "mut" on the callable
};

; 3. Module-level externs
@Mod:puts extern ccc "puts" (*u8) -> (i32);

; 4. Module-level helpers
@Mod:helper input { :n u64; } -> {
    log_d(input:n);
};

; 5. Entry point
create() -> {
    inst := MyType;
    inst:do_thing { :x u64 42; };
    mod:puts("done\n");
}
```

Appendix C – Screenshots

Appendix D – Source Code

Included in this appendix is a snapshot of the project repository as of the date of submission of this interim report. The full repository, including the entire git history, is available upon request. Please request access to the private repository located at https://github.com/davidcallanan/mu-fyp.

Build artifacts and other uninteresting files have been omitted from this appendix. This includes, but is not limited to, all files mentioned inside .gitignore.

Appendix E – LLM Prompts

Appendix F – Interim Report

FYP Interim Report 2025-2026
Department of Computer Science

Student Name	David P. Callanan	**Student Number**	21444104
Supervisor	Dr. Phil Maguire	**ECTS Credits**	5
Project Title	Studying link-time optimizations in programming language development to facilitate the continuum of static and dynamic modules.		

Contents

1 Project Objectives **2**

2 Description of Work Completed **2**

2.1 Evidence of Work Completed 3

2.2 Literature Review 3

2.3 Use of GenAI and Tools 3

3 Future Work **4**

A Appendix: Screenshots **6**

B Appendix: Source Code **9**

C Appendix: LLM Prompts **75**

1 Project Objectives

The goal of this project is to develop a proof-of-concept programming language ("Essence C") that facilitates bespoke optimization techniques via novel language constructs. The specific objective is to research the continuum of static and dynamic modules in low-level software development. A module system will be designed to promote dispatch flexibility by abstracting away the underlying dispatch mechanism, enabling aggressive link-time optimizations when circumstances allow. The development of a prototype compiler will demonstrate the efficacy of the proposed solution.

Success will be evidenced by the following deliverables:

(1) A functional compiler that implements the designed module system;

(2) Documentation of the module system and other core language features;

(3) A simple IDE extension that provides syntax highlighting to the programmer;

(4) A body of research into the continuum of static and dynamic modules in software development, with kernel development as a case study;

(5) An implementation of link-time optimization techniques that leverage the designed module system;

(6) An evaluation of the effectiveness of these optimizations through the benchmarking of sample programs;

(7) A monorepo containing extensive git history, demonstrating ongoing development progress;

(8) A final written report detailing the research, design, implementation and evaluation of the project.

Some considerations will be out of scope for this project, such as the performance of compilation, the level of detail of compilation errors, and the implementation of standard library functions and advanced type system features.

2 Description of Work Completed

Preliminary work on syntax design and language parsing code has been completed, and background research into the LLVM toolchain has been conducted. AI assistance was also briefly assessed.

2.1 Evidence of Work Completed

Initially, a handful of pseudocode files were created to brainstorm syntax ideas for the language. Then, a simple VS Code extension was developed to provide syntax highlighting for these early syntax ideas to reduce eye strain. Next, work commenced on the compiler frontend, written in JavaScript. Some previously written code from personal projects was used (including a simple parsing library), justifying the decision to use JavaScript. Basic parse-tree generation was implemented for a portion of the preliminary syntax, and a custom "source scope" system was developed to handle the translation between filesystem resources and source references within the programming language. Then, the build system for the compiler backend (written in C++) was organized. A Makefile was used within a Dockerized environment, itself orchestrated by a Linux/WSL environment for best reproducibility. C++ was justified due to its first-class integration with the LLVM toolchain [1].

2.2 Literature Review

LLVM was found to be a practical compilation target for low-level programming languages due to (1) its target-independent code generation [2] and (2) its integration capabilities with existing programming languages thanks to ABI interoperability (including some recent improvements [3]). Furthermore, the extensive optimization passes shared amongst languages targetting LLVM [4] should make *Essence C* directly comparable with existing languages, allowing benchmarks to be solely influenced by the additional optimization techniques employed. For instance, C can be compiled with LLVM via the popular Clang compiler [5]. It was also noticed that LLVM has experimental support for inline assembly [6], preventing the layer of indirection of a separate compilation unit (useful in kernel development [7]). Additionally, LLVM benefits greatly from intermodular optimizations at link-time [8], because its intermediate representation (IR) preserves the necessary higher-level constructs. Critically, the LLVM API exposes a powerful interface to programatically manipulate existing LLVM IR files [9] [10]. It follows that inlined compatibility wrappers can be generated to minimize optimization barriers between compilation units originating from languages other than *Essence C*.

2.3 Use of GenAI and Tools

Two notable GenAI tools have been used to assist with the research and development of this project so far.

(1) The first is t3.chat, a web application that consolidates numerous large language

models into a streamlined chat interface [11]. The user can experiment with different models to understand which models are best suited for the task at hand. Both research and programming tasks have benefited from the use of this tool. The tool was primarily used to steer the work in the right direction, and to identify the concepts, libraries and resources through which further research could be conducted. For transparency, all LLM prompts have been included in **Appendix C**.

(2) The second is the tab-complete feature of GitHub Copilot. This tool has encouraged rapid development and iteration through its behaving as a glorified autocomplete. From experience, the tool's context is severely limited, forcing the developer to have genuine knowledge of the codebase and its architecture. This is beneficial for projects that require significant unique input from the developer, in contrast to rehash projects that benefit more directly from LLM assistance.

Some experimentation with GitHub Copilot agent mode is ongoing, with notable issues identified. Firstly, the development speed is impeded by slow model response times, lack of funds, and internet connectivity issues. Secondly, there is a great difficulty in referencing contributions made by this tool due to (1) tool calls that modify the codebase directly and (2) significant implicit pollution of the context window as the agent navigates the respository. As such, usage of this tool is temporarily suspended pending further assessment.

3 Future Work

The core of the project revolves around the design of the module system. The accompanying research will be a significant undertaking, including static vs dynamic dispatch, module instance tracking, constraint propagation, and the feasibility of IR interoperability persuant to the proposed optimization goals. This research will be tackled first, in parallel with the development of any necessary boilerplate code for the compiler backend. Then, some experimentation will be needed with LLVM IR generation before continuing with the compiler backend. Everything technical other than the compiler backend is expected to be completed by the end of January, leaving adequate time for the implementation and evaluation of the compiler proper.

References

[1] The LLVM Project. "Frequently asked questions (FAQ)," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/FAQ.html#in-what-language-is-llvm-written

[2] The LLVM Project. "The LLVM target-independent code generator," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/CodeGenerator.html

[3] N. Sreekumar. "GSoC 2025: Introducing an ABI lowering library," Accessed: Nov. 7, 2025. [Online]. Available: https://blog.llvm.org/posts/2025-08-25-abi-library/

[4] The LLVM Project. "LLVM's analysis and transform passes," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/Passes.html#domtree-dominator-tree-construction

[5] The Clang Project. "Clang: C language family frontend for LLVM," Accessed: Nov. 17, 2025. [Online]. Available: https://clang.llvm.org/

[6] The LLVM Project. "LLVM language reference manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LangRef.html#inline-assembler-expressions

[7] D. Roy. "GCC inline assembly and its usage in the linux kernel," Accessed: Nov. 6, 2025. [Online]. Available: https://dl.acm.org/doi/fullHtml/10.5555/3024956.3024958

[8] The LLVM Project. "LLVM link time optimization: Design and implementation," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/LinkTimeOptimization.html

[9] The LLVM Project. "LLVM programmer's manual," Accessed: Nov. 17, 2025. [Online]. Available: https://llvm.org/docs/ProgrammersManual.html

[10] The LLVM Project. "Writing an LLVM pass," Accessed: Nov. 2, 2025. [Online]. Available: https://llvm.org/docs/WritingAnLLVMNewPMPass.html

[11] DigiVirus. "Meet T3 chat AI: Your all-in-one, super-fast AI assistant," Accessed: Nov. 16, 2025. [Online]. Available: https://digivirus.in/meet-t3-chat-ai-your-all-in-one-super-fast-ai-assistant/